

BUSCANDO AL CORONEL KURTZ

**O SEA, ¿CÓMO QUE MI ALGORITMO A* ENCUENTRA CAMINOS
MÁS RAPIDO QUE YO ENCONTRANDO MIS LLAVES DE CASA?**

FUNDAMENTOS DE LA INTELIGENCIA ARTIFICIAL

ÍNDICE

INTRODUCCIÓN	3
DISEÑO E IMPLEMENTACIÓN.....	5
MANUAL DE USUARIO	29
AUTORÍA.....	37
BIBLIOGRAFÍA.....	38
ANEXO.....	39

INTRODUCCIÓN

Este documento, realizado por Claudia Maria Lopez Bombin, alumna de 2B del Grado en Ingeniería Matemática e Inteligencia Artificial en la Universidad Pontificia Comillas, corresponde a la memoria complementaria al proyecto final de la asignatura Fundamentos de la Inteligencia Artificial.

Los fundamentos de la búsqueda en grafos y la planificación bajo incertidumbre que sustentan los algoritmos implementados en este proyecto encuentran sus raíces en los principios establecidos por la investigación pionera en inteligencia artificial. La representación de problemas como espacios de estados, fundamental para la navegación autónoma del Capitán Willard, fue formalizada por Newell y Simon en su teoría del espacio-problema, donde demostraron cómo cualquier tarea de resolución de problemas puede modelarse como un grafo de estados conectados por operadores (Newell & Simon, 1976).

Lo que inicialmente se conceptualizó como una exploración sistemática de espacios de búsqueda se ha transformado, gracias a avances como los algoritmos de búsqueda heurística (Pearl, 1984), en el mecanismo esencial que permite a agentes artificiales navegar entornos complejos bajo incertidumbre. Como señala Russell en su análisis de agentes basados en objetivos, "la búsqueda proporciona un marco unificado para la resolución de problemas en espacios discretos" (Russell & Norvig, 1995), principio que aplico directamente en la exploración del palacio.

Los algoritmos implementados, desde BFS para exploración sistemática hasta A* con heurísticas basadas en distancias Manhattan, se basan en propiedades fundamentales de la optimalidad de búsqueda informada demostrada por Hart, Nilsson y Raphael. Su algoritmo A*, utilizado en nuestra implementación para calcular caminos seguros hacia Kurtz, garantiza optimalidad cuando la heurística es admisible, cumpliendo precisamente con los criterios establecidos en su publicación seminal.

La inferencia bayesiana implementada en la segunda parte del proyecto ejemplifica cómo el razonamiento probabilístico actualiza creencias ante nueva evidencia, principio formalizado por Bayes y aplicado sistemáticamente en sistemas de diagnóstico y navegación probabilística. Como destaca Pearl en su trabajo sobre redes bayesianas, "la actualización de probabilidades condicionadas a evidencias permite inferencias robustas bajo incertidumbre" (Pearl, 1988), mecanismo que nuestro agente utiliza para deducir ubicaciones de trampas a partir de perceptos.

Finalmente, la implementación de Value Iteration para el cruce del río aplica directamente los principios de los Procesos de Decisión de Markov (MDP) formalizados por Bellman, cuya ecuación fundamental, la ecuación de Bellman, proporciona el marco para optimizar políticas en entornos estocásticos. Como demostraron Sutton y Barto, estos métodos constituyen la base del aprendizaje por refuerzo moderno, permitiendo que nuestro agente balancee explotación de conocimiento con exploración de nuevas rutas.

La transformación de estos conceptos teóricos en agentes funcionales —desde la búsqueda lógica hasta la planificación probabilística— demuestra cómo los fundamentos matemáticos de la IA pueden materializarse en comportamientos inteligentes y adaptativos, validando la observación de Simon sobre sistemas adaptativos complejos: "La inteligencia artificial no se trata de replicar la mente humana, sino de crear sistemas que exhiban comportamiento inteligente" (Simon, 1996).

En el contexto de los sistemas autónomos de navegación y planificación bajo incertidumbre se desarrolla este proyecto, cuyo objetivo principal es implementar múltiples agentes inteligentes mediante la aplicación de

algoritmos de búsqueda, inferencia probabilística y procesos de decisión de Markov, demostrando su utilidad práctica en el ámbito de la exploración de entornos hostiles con información parcial y afianzando los fundamentos matemáticos que sustentan la toma de decisiones secuenciales en espacios de estados complejos.

Como objetivos secundarios de este proyecto encontramos:

- Desarrollar agentes inteligentes en Python utilizando múltiples paradigmas (búsqueda, inferencia bayesiana, MDPs).
- Implementar algoritmos eficientes de exploración y planificación en entornos con incertidumbre.
- Crear interfaces interactivas para simular y observar el comportamiento de los agentes.
- Incorporar mecanismos robustos de gestión de errores y prevención de bucles.
- Validar el rendimiento mediante pruebas automatizadas con configuraciones aleatorias.
- Integrar diferentes enfoques de IA en una arquitectura de software coherente y comparable.
- Conectar la teoría de IA con su implementación práctica mediante código funcional.
- Desarrollar visualizaciones que expliquen el proceso de decisión de cada agente.

El proyecto consiste en la implementación de tres agentes inteligentes para navegación autónoma, compuestos por tres módulos .py principales:

1. **kurtz.py** (Parte 1): implementa un agente lógico que utiliza algoritmos de búsqueda (BFS, heurísticas) para explorar sistemáticamente el palacio, evitando precipicios y al soldado enemigo mientras busca al coronel Kurtz.
2. **palacio_bayesiano.py** (Parte 2): implementa un agente bayesiano que aplica inferencia probabilística para actualizar creencias sobre la ubicación de trampas específicas (fuego, pinchos, dardos) a partir de perceptos diferenciados.
3. **river_mdp.py** (Parte 2): implementa un proceso de decisión de Markov (MDP) con Value Iteration para calcular políticas óptimas en el entorno estocástico del río, balanceando riesgo y eficiencia en el cruce.

Estos módulos se integran mediante el programa **main.py**, que ofrece un menú unificado para ejecutar y comparar los diferentes enfoques, y **funciones_comunes.py**, que proporciona utilidades compartidas y algoritmos base

El sistema integra múltiples paradigmas de inteligencia artificial para proporcionar una solución completa de navegación autónoma en entornos hostiles e inciertos, demostrando la aplicación práctica de la teoría de agentes a problemas reales de exploración y planificación bajo incertidumbre.

DISEÑO E IMPLEMENTACIÓN

1. Arquitectura del código

La arquitectura del sistema implementa una estructura modular que establece una clara división de responsabilidades entre componentes especializados. Este enfoque facilita el mantenimiento, el aislamiento de fallos y la escalabilidad. Cada módulo opera con autonomía funcional, permitiendo optimizaciones específicas sin comprometer la estabilidad global.

El sistema se articula alrededor de tres pilares fundamentales que interactúan de forma coordinada:

a. kurtz.py: el núcleo algorítmico de navegación lógica

El archivo **kurtz.py** constituye el núcleo algorítmico de la parte 1 del sistema, implementando los mecanismos esenciales para la navegación lógica y la exploración sistemática en entornos discretos. Este módulo especializado proporciona las herramientas computacionales necesarias para resolver problemas de búsqueda con restricciones de seguridad, basándose en los siguientes pilares algorítmicos:

- Implementación de algoritmos de búsqueda clásica

Sistema fundamental para la exploración sistemática del palacio, implementando estrategias como búsqueda en amplitud (BFS) y búsqueda heurística informada. El agente maneja eficientemente tanto la exploración exhaustiva como la dirigida por objetivos, adaptándose a las restricciones de seguridad impuestas por precipicios y el soldado enemigo.

- Arquitectura de agente lógico basado en conocimiento

Implementación de un sistema de inferencia que actualiza dinámicamente el conocimiento sobre celdas seguras, peligrosas y visitadas. El agente deduce la posible ubicación de peligros a partir de perceptos como brisa (indicando precipicios adyacentes) y ronquido (indicando proximidad al soldado), aplicando razonamiento lógico para tomar decisiones informadas.

- Mecanismos anti-bucle y exploración adaptativa

Sistema robusto de detección y recuperación de ciclos que previene que el agente quede atrapado en patrones repetitivos. Incluye estrategias de escape, contadores de intentos por dirección y exploración forzada de áreas no visitadas cuando se detectan bucles.

El procesamiento de la navegación del agente sigue una secuencia definida por:

1. Inicialización y validación del entorno: verifica la correcta generación del palacio con todos los elementos (precipicios, soldado, Kurtz, salida) en posiciones válidas y únicas, asegurando la consistencia del estado inicial antes de iniciar cualquier exploración.
2. Ejecución cíclica de percepción-decisión-acción: aplicación del ciclo básico de agentes donde en cada paso: (a) se recogen perceptos del entorno actual, (b) se actualiza el conocimiento lógico, (c) se decide la siguiente acción óptima basada en el conocimiento acumulado, (d) se ejecuta la acción y se observan sus consecuencias.
3. Gestión dinámica del conocimiento: actualización continua del mapa mental del agente, marcando celdas como visitadas, seguras o peligrosas según la evidencia recogida, y ajustando las estimaciones de riesgo para celdas no exploradas.

-

El diseño de kurtz.py incorpora técnicas de optimización algorítmica y arquitectónica para establecer una base sólida que garantice eficiencia, mantenibilidad y robustez en el sistema de navegación autónoma:

- b. `palacio_bayesiano.py`: el núcleo algorítmico de navegación probabilística

El archivo **palacio_bayesiano.py** constituye el núcleo probabilístico del sistema de navegación, implementando los algoritmos esenciales para el cálculo de riesgos óptimos y la toma de decisiones bajo incertidumbre. Esta librería especializada proporciona las herramientas computacionales necesarias para

resolver problemas de inferencia en entornos con múltiples fuentes de incertidumbre, basándose en los siguientes pilares algorítmicos:

- Implementación de inferencia bayesiana

Algoritmo fundamental para la actualización de creencias probabilísticas ante nueva evidencia, optimizado para entornos con múltiples tipos de peligros diferenciados. El sistema maneja eficientemente tanto evidencias positivas (detectar un estímulo) como negativas (no detectarlo), adaptándose a las características específicas de cada tipo de trampa.

- Sistema de distribuciones probabilísticas múltiples

Implementación de mapas de probabilidad separados para cada elemento del entorno (trampa de fuego, pinchos, dardos, soldado, salida), proporcionando representaciones robustas para problemas de localización en espacios complejos y permitiendo el análisis diferenciado de diferentes tipos de amenazas.

- Arquitectura flexible de umbrales de riesgo

Sistema de evaluación de riesgos parametrizable que permite adaptar las decisiones a diferentes criterios de seguridad, facilitando la implementación de estrategias que balancean exploración de áreas desconocidas con evitación de peligros probables según la tolerancia al riesgo configurada.

- Mecanismos de actualización incremental

Estructura que permite la incorporación progresiva de nueva información sensorial, actualizando las distribuciones de probabilidad de manera eficiente sin necesidad de recalcular desde cero en cada paso, optimizando el rendimiento computacional durante la exploración continua del palacio.

- Sistema integrado de detección y recuperación de bucles

Implementación de estrategias avanzadas para identificar patrones cíclicos en el comportamiento del agente y activar mecanismos de escape específicos, incluyendo exploración forzada y aumento temporal de la tolerancia al riesgo cuando es necesario para romper ciclos improductivos.

El procesamiento del agente bayesiano sigue una secuencia definida por:

1. Inicialización y configuración del sistema probabilístico: establecimiento de distribuciones a priori uniformes para todos los elementos del entorno, configuración de los umbrales de riesgo aceptables, y verificación de la consistencia del palacio generado antes de iniciar la exploración.
2. Ciclo de percepción-actualización-decisión: ejecución del flujo básico del agente donde en cada paso: (a) se recogen perceptos específicos de cada tipo de trampa, (b) se aplica la regla de Bayes para actualizar cada distribución de probabilidad, (c) se calcula el riesgo total por celda, (d) se decide la siguiente acción basándose en el riesgo calculado y las estrategias activas.
3. Gestión de múltiples distribuciones de probabilidad: mantenimiento y actualización simultánea de los mapas de creencia para cada tipo de peligro y elemento, asegurando que las inferencias sobre diferentes amenazas se mantengan separadas pero se integren adecuadamente en la evaluación de riesgo total.

4. Detección y respuesta a situaciones especiales: identificación de condiciones que requieren estrategias específicas, como bucles en la trayectoria, falta de celdas seguras disponibles, o proximidad a objetivos importantes (Kurtz o salida), activando los mecanismos correspondientes.
5. Formateo y visualización de información probabilística: transformación de las distribuciones internas de probabilidad en representaciones comprensibles para el usuario, incluyendo mapas de calor, estadísticas de riesgo, y análisis comparativos de diferentes configuraciones de umbral.



Figura 2. Diagrama de flujo del programa palacio_bayesiano.py. Elaboración propia.

El diseño de palacio_bayesiano.py incorpora técnicas de optimización algorítmica y arquitectónica para establecer una base sólida que garantice eficiencia, mantenibilidad y robustez en el sistema de inferencia probabilística:

- Inferencia bayesiana optimizada: implementamos versiones eficientes de actualización de probabilidades mediante selección inteligente de estructuras de datos y aprovechamiento de propiedades matemáticas de la regla de Bayes, como la normalización incremental de distribuciones.
- Detección precoz de bucles: identificamos tempranamente patrones cíclicos en el comportamiento del agente analizando el historial de movimientos, evitando exploración repetitiva que consume recursos computacionales sin generar nuevo conocimiento.
- Validación progresiva de creencias: aplicamos verificaciones continuas que ajustan las distribuciones de probabilidad en cuanto se recibe nueva evidencia, optimizando el proceso de inferencia y mejorando la calidad de las decisiones del agente.
- Manejo eficiente de distribuciones múltiples: utilizamos estructuras de datos especializadas para mantener separados los mapas de probabilidad de diferentes elementos, minimizando la sobrecarga de memoria durante el procesamiento de las múltiples fuentes de incertidumbre.
- Gestión adaptativa de umbrales de riesgo: implementamos mecanismos que ajustan dinámicamente la tolerancia al riesgo según las circunstancias, permitiendo mayor agresividad en exploración cuando se detectan bucles o menor tolerancia cuando se acerca a objetivos críticos.

c. river_mdp.py: el núcleo algorítmico de planificación óptima

El archivo river_mdp.py constituye el núcleo matemático del sistema de planificación óptima, implementando los algoritmos esenciales para el cálculo de políticas óptimas en procesos de decisión de Markov. Esta librería especializada proporciona las herramientas computacionales necesarias para resolver problemas de optimización en entornos estocásticos, basándose en los siguientes pilares algorítmicos:

- Implementación de Value Iteration

Algoritmo fundamental para el cálculo de valores de estado óptimos en MDPs con factor de descuento, optimizado para entornos con transiciones probabilísticas. El sistema maneja eficientemente tanto efectos deterministas como estocásticos, adaptándose a las características reales del entorno del río con corrientes variables y obstáculos.

- **Modelado de transiciones probabilísticas**

Implementación de la función de transición que calcula las probabilidades de llegar a diferentes estados desde cada estado-acción, proporcionando representaciones robustas para problemas de decisión en entornos inciertos y permitiendo el análisis de las consecuencias esperadas de cada acción disponible.

- **Sistema flexible de recompensas**

Arquitectura de funciones de recompensa parametrizables que permite adaptar los cálculos a diferentes criterios de optimización (llegar a salida, evitar trampas, minimizar pasos), facilitando la implementación de políticas que balancean múltiples objetivos contrapuestos según su importancia relativa.

- **Cálculo de políticas óptimas**

Proceso que transforma los valores de estado calculados mediante Value Iteration en una política concreta que especifica la mejor acción para cada posición posible, considerando tanto recompensas inmediatas como consecuencias futuras descontadas.

- **Simulación de trayectorias estocásticas**

Mecanismo para ejecutar múltiples recorridos siguiendo la política óptima, incorporando los efectos aleatorios del entorno para evaluar estadísticamente el desempeño esperado de la estrategia calculada bajo diferentes realizaciones de la incertidumbre.

El procesamiento del MDP sigue una secuencia definida por:

1. Inicialización y configuración del entorno estocástico: generación del río con islas aleatorias, cálculo de fuerzas de corriente por columna, verificación de la accesibilidad del espacio de estados, y configuración de los parámetros del algoritmo (factor de descuento gamma, criterio de convergencia epsilon) antes de iniciar Value Iteration.
2. Ejecución de Value Iteration optimizada: aplicación iterativa de la ecuación de Bellman utilizando estructuras de datos eficientes para representar valores de estado y políticas, actualizando progresivamente las estimaciones hasta alcanzar convergencia dentro del umbral establecido.
3. Cálculo de transiciones probabilísticas: evaluación sistemática de las probabilidades de transición para cada par estado-acción, considerando los efectos combinados del movimiento deseado, el empuje de la corriente, y las colisiones con obstáculos o bordes.
4. Extracción y validación de la política óptima: transformación de los valores de estado convergidos en una política concreta que especifica la mejor acción para cada posición, verificando la consistencia interna y optimalidad de las decisiones resultantes.
5. Simulación y evaluación de desempeño: ejecución de múltiples trayectorias siguiendo la política óptima, incorporando los efectos aleatorios del entorno para calcular métricas estadísticas de éxito, longitud promedio de camino, y recompensa esperada acumulada.
6. Formateo y visualización de resultados: transformación de los valores de estado, políticas y estadísticas de simulación en representaciones comprensibles, incluyendo mapas de valores, visualizaciones de política con símbolos direccionales, y análisis comparativos de diferentes configuraciones del entorno.



Figura 3. Diagrama de flujo del programa river_mdp.py. Elaboración propia.

El diseño de river_mdp.py incorpora técnicas de optimización algorítmica y arquitectónica para establecer una base sólida que garantice eficiencia, mantenibilidad y robustez en el sistema de planificación óptima:

- Value Iteration optimizado: implementamos versiones eficientes del algoritmo mediante selección inteligente de estructuras de datos y aprovechamiento de propiedades matemáticas de los MDPs, como la convergencia garantizada de las iteraciones de la ecuación de Bellman bajo factor de descuento.
- Detección precoz de estados terminales: identificamos tempranamente configuraciones inválidas como estados fuera del espacio o transiciones imposibles, evitando procesos de cálculo costosos cuando están condenados al error en la simulación de trayectorias.
- Validación progresiva de políticas: aplicamos verificaciones continuas que garantizan la coherencia interna de la política calculada, optimizando el tiempo de procesamiento y mejorando la confiabilidad del sistema de planificación.
- Manejo eficiente del espacio de estados: utilizamos estructuras de datos especializadas como matrices numpy para valores y políticas, minimizando la sobrecarga de memoria durante el procesamiento de todos los estados posibles en el entorno del río.
- Gestión adaptativa de parámetros: implementamos mecanismos que ajustan dinámicamente factores como la fuerza de corriente y ubicación de islas según configuraciones aleatorias, permitiendo evaluar el desempeño del algoritmo bajo diferentes condiciones de dificultad.

2. Algoritmos y clases principales

El núcleo del sistema de agentes inteligentes implementado reside en la aplicación eficiente de algoritmos fundamentales de la inteligencia artificial y la teoría de la decisión. Cada funcionalidad ha sido programada con un enfoque dual: profundizar en sus bases teóricas para garantizar corrección conceptual, mientras se exploran optimizaciones algorítmicas que mantienen la viabilidad computacional en tiempo real. El diseño prioriza la aplicabilidad práctica en escenarios de navegación autónoma bajo incertidumbre, donde el balance entre precisión inferencial y eficiencia de decisión resulta crítico.

En este apartado analizaremos exhaustivamente cada algoritmo y clase implementado, justificando las decisiones de diseño tomadas en función de propiedades matemáticas específicas de cada paradigma, así como el flujo de ejecución que asegura tanto la robustez en la exploración de entornos como la eficiencia operativa del sistema completo de agentes:

a. clase Palacio:

La clase Palacio constituye el núcleo ambiental de la Parte 1 del proyecto, implementando un entorno discreto y determinista donde el Capitán Willard debe navegar para cumplir su misión. Esta clase representa no solo el escenario físico del problema, sino también el modelo computacional que hace posible la interacción entre el agente inteligente y su mundo simulado. Su diseño sigue principios establecidos de sistemas multiagente donde el entorno proporciona percepciones, recibe acciones y evoluciona su estado de manera predecible

El palacio se define como una cuadrícula cuadrada de dimensiones $n \times n$, siendo $n = 6$ por defecto. Esta elección establece un espacio de estados manejable (36 celdas) pero suficientemente complejo para requerir estrategias de exploración no triviales. La posición del capitán se inicializa sistemáticamente en la coordenada (0, 0), estableciendo un punto de partida conocido y consistente para todas las simulaciones.

La clase gestiona tres categorías de elementos posicionales:

1. **Peligros estáticos:** Tres precipicios ubicados en posiciones aleatorias pero mutuamente exclusivas.
2. **Amenazas activas:** Un soldado enemigo que representa un peligro condicional a su estado de vitalidad.
3. **Objetivos estratégicos:** El coronel Kurtz como objetivo primario y la salida como objetivo secundario condicionado.

La implementación mantiene tres estructuras de conocimiento paralelas:

- **celdas_visitadas:** Registro histórico de exploración que crece monótonicamente.
- **celdas_seguras:** Conjunto de posiciones verificadas como libres de peligro.
- **celdas_peligrosas:** Espacio para marcar hipótesis de riesgo (actualmente subutilizado).

Este triplete permite al agente razonar sobre lo conocido, lo inferido y lo desconocido, implementando una forma básica de epistemología computacional.

El proceso de inicialización sigue una secuencia rigurosa:

1. **Generación del espacio base:** Se crea una lista exhaustiva de todas las celdas excepto la posición inicial, garantizando que el agente nunca comience sobre un elemento peligroso.
2. **Aleatorización completa:** Mediante `random.shuffle()` se obtiene una permutación uniformemente aleatoria del espacio disponible.
3. **Asignación secuencial:** Los elementos se extraen en orden predefinido: precipicios primero, luego soldado, Kurtz y finalmente salida. Este orden asegura prioridades de ubicación y evita conflictos posicionales.

El espacio de configuraciones posibles es considerable:

- Celdas disponibles para asignación: 30 (36 totales - 1 posición inicial - 5 elementos)
- Combinaciones de precipicios: $\binom{30}{3} = 4,060$
- Disposiciones completas: millones de configuraciones únicas

Esta diversidad garantiza que las pruebas no se reduzcan a casos particulares sino que evalúen la robustez del agente frente a arreglos arbitrarios.

El método mover implementa un sistema de transiciones de estado que verifica:

1. **Viabilidad geométrica:** El movimiento propuesto debe mantenerse dentro de los límites del tablero.
2. **Seguridad inmediata:** La celda destino no puede contener precipicios ni al soldado vivo.
3. **Actualización del conocimiento:** Las estructuras de conocimiento se modifican para reflejar la nueva información.

Cada movimiento exitoso desencadena una cascada de verificaciones:

- **Detección de Kurtz:** Si se alcanza la posición de Kurtz, se activa el estado `con_kurtz`, modificando los objetivos subsiguientes.
- **Verificación de salida:** La llegada a la salida solo es significativa si se acompaña de Kurtz, implementando así la condición de victoria compuesta.
- **Registro exploratorio:** Se actualizan las listas de celdas conocidas, manteniendo un historial completo del recorrido.

La función `que_siento` genera un conjunto discreto de señales:

Categoría	Perceptos	Condición de Activación	Significado Semántico
Restricciones físicas	<code>pared_norte/sur/oeste/este</code>	Posición en borde correspondiente	Límite del espacio navegable
Indicadores de peligro	<code>brisa</code>	Precipicio en celda adyacente	Riesgo de caída inminente
	<code>ronquido</code>	Soldado vivo en celda adyacente	Amenaza de confrontación mortal
Señales de objetivo	<code>resplandor_fuerte</code>	En celda de salida	Objetivo final alcanzado
	<code>resplandor_suave</code>	Salida en celda adyacente	Proximidad al objetivo final

Observamos lo siguiente:

1. **Localidad estricta:** Los perceptos dependen exclusivamente de la celda actual y sus vecinos inmediatos.
2. **Determinismo completo:** Misma situación ambiental produce mismos perceptos.
3. **Complejidad informativa:** Incluye toda evidencia disponible para inferencia.
4. **Ausencia de ruido:** No hay percepciones erróneas o ambiguas.

El método `vecinos_de` implementa la noción de adyacencia según la distancia de Manhattan, definiendo el espacio de movimientos posibles desde cualquier posición. Esta función sirve como componente fundamental tanto para la generación de perceptos como para la planificación de rutas.

La clase incluye dos modos de visualización:

1. **Mapa conocido:** Muestra solo la información descubierta por el agente, reflejando su conocimiento parcial del entorno.
2. **Mapa completo (cheat):** Revela todas las posiciones independientemente del descubrimiento, útil para análisis y depuración.

La simbología empleada ([CW], [CK], [P], [S], [X], [K], [E], [·], [v], [?]) crea un lenguaje visual intuitivo que comunica eficientemente el estado complejo del sistema.

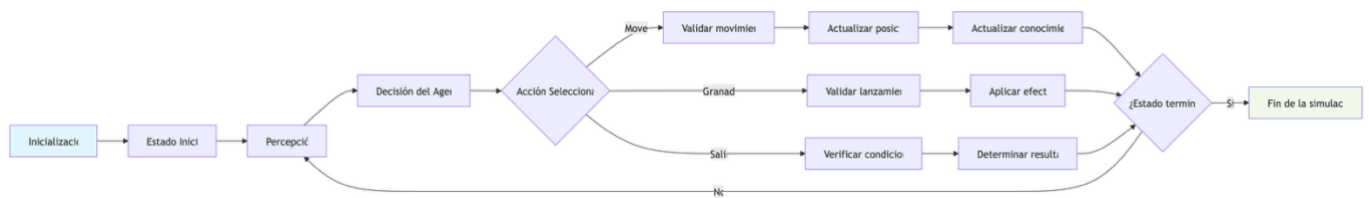


Figura 4. Diagrama de flujo global de la clase Palacio. Elaboración propia.

NOTA: En el anexo se encuentran los diagramas de flujo de cada una de las funciones contenidas dentro de la clase de manera detallada.

b. class Agente explorador:

La clase AgenteExplorador representa la implementación central del agente lógico en la Parte 1 del proyecto, diseñada para navegar autónomamente el entorno definido por la clase Palacio. Este agente implementa un sistema híbrido de búsqueda y deducción que combina exploración sistemática con razonamiento lógico sobre peligros inferidos.

El agente mantiene una representación estructurada del conocimiento a través de un diccionario multidimensional:

```

1  # Inicializar conocimiento (matriz de conocimiento)
2  # Esto es un diccionario gigante. No es eficiente pero funciona. Se xq? no, pero funciona
3  for i in range(self.p.n):
4      for j in range(self.p.n):
5          self.conocimiento[(i, j)] = {
6              'visitada': False,
7              'segura': False,
8              'precipicio': False,
9              'soldado': False,
10             'brisa_detectada': False,
11             'ronquido_detectado': False,
12             'riesgo_precipicio': 0, # Contador de riesgo precipicio
13             'riesgo_soldado': 0,   # Contador de riesgo soldado
14             'resplandor': False   # Nuevo: si hemos sentido resplandor
15         }
  
```

Figura 5. Código de inicio de estructura del conocimiento inicial. Elaboración propia.

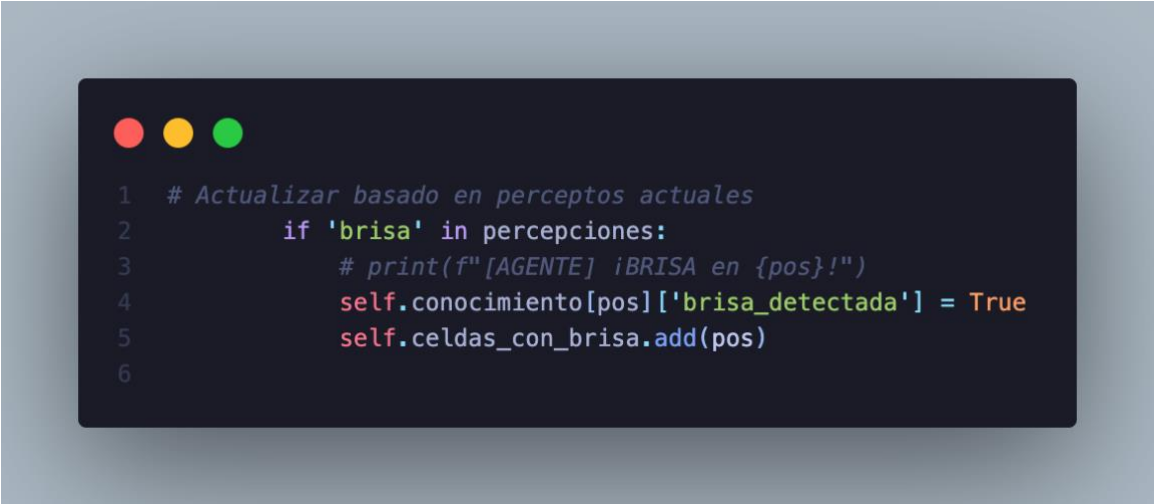
Esta estructura permite un razonamiento diferenciado sobre múltiples dimensiones del entorno, implementando una forma básica de lógica de primer orden donde cada celda tiene propiedades independientes pero relacionadas.

Observamos también la existencia de sistintos componentes d seguridad y control del código:

- self.stack: Pila para exploración sistemática tipo DFS.
- self.camino: Historial completo del recorrido para análisis y prevención de bucles.

- self.celdas_con_brisa / self.celdas_con_ronquido: Conjuntos para inferencia lógica.
- self.intentos_movimiento: Diccionario para evitar repeticiones improductivas.
- self.objetivo_actual: Meta específica en cada momento.
- self.camino_a_salida: Camino óptimo precalculado cuando se detecta la salida.

El método actualizar_conocimiento() implementa un motor de inferencia que transforma perceptos crudos en conocimiento estructurado:



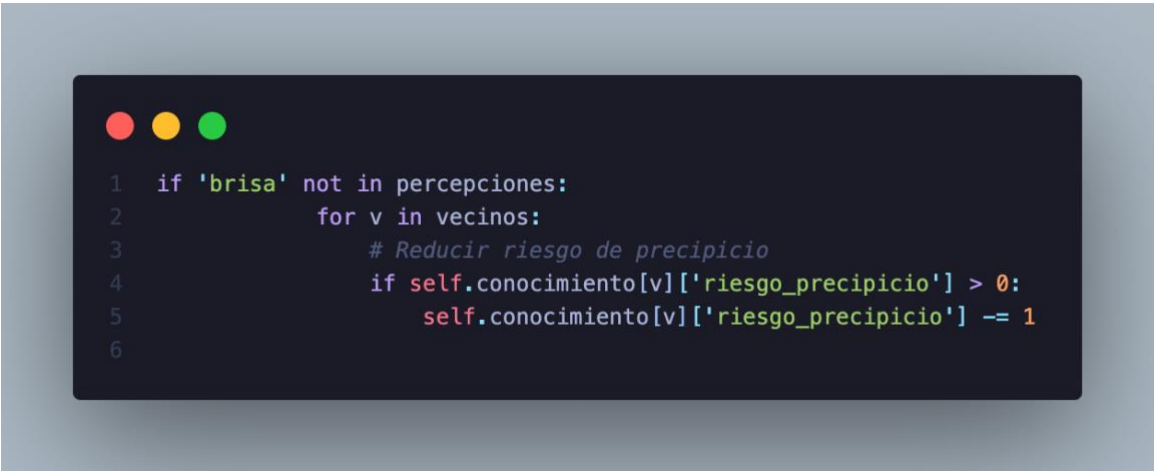
```

1  # Actualizar basado en perceptos actuales
2      if 'brisa' in percepciones:
3          # print(f"[AGENTE] ¡BRISA en {pos}!")
4          self.conocimiento[pos]['brisa_detectada'] = True
5          self.celdas_con_brisa.add(pos)
6

```

Figura 6. Código de ejemplo de actualización de perceptos por evidencia positiva. Elaboración propia.

Este mecanismo implementa razonamiento por casos acumulativos: múltiples detecciones de un percepto desde diferentes posiciones refuerzan la hipótesis correspondiente, mientras que su ausencia la debilita. Además, el agente utiliza tanto evidencia positiva (presencia de perceptos) como evidencia negativa (ausencia de perceptos esperados):



```

1  if 'brisa' not in percepciones:
2      for v in vecinos:
3          # Reducir riesgo de precipicio
4          if self.conocimiento[v]['riesgo_precipicio'] > 0:
5              self.conocimiento[v]['riesgo_precipicio'] -= 1
6

```

Figura 7. Código de ejemplo de actualización de perceptos por evidencia negativa. Elaboración propia.

Este enfoque permite refutar hipótesis incorrectas y ajustar dinámicamente las creencias del agente, implementando una forma primitiva de aprendizaje por corrección de error.

El método `decidir_siguiente_movimiento()` implementa una jerarquía de 10 estrategias evaluadas en orden de prioridad:

Prioridad	Estrategia	Condición de Activación	Objetivo
1	Salida inmediata	En salida con Kurtz	Terminar misión
2	Uso de granada	Ronquido detectado + granada disponible	Eliminar amenaza
3	Celdas seguras no visitadas	Existen celdas seguras desconocidas	Exploración segura
4	Objetivo actual establecido	Hay objetivo predefinido	Navegación dirigida
5	Nuevo objetivo cercano	Celdas no visitadas accesibles	Exploración expansiva
6	Retroceso estratégico	Sin opciones hacia adelante	Recuperación de posición
7	Movimiento conservador	Direcciones seguras disponibles	Exploración cautelosa
8	Escape de bucle	Patrón cíclico detectado	Romper comportamiento repetitivo
9	Movimiento arriesgado	Sin peligros inmediatos	Exploración agresiva
10	Último recurso	Todas las demás fallaron	Prevenir bloqueo total

El agente implementa una máquina de estados de objetivos:

- 1. **Fase de búsqueda de Kurtz:** Prioriza exploración exhaustiva.
 - 2. **Fase de búsqueda de salida** (tras encontrar Kurtz): Prioriza navegación eficiente.
 - 3. **Fase de escape** (detectado bucle): Prioriza romper patrones.
- En cuanto a los mecanismos de anti-bucle, debemos de destacar la detección de patrones cíclicos:

```
1 def detectar_bucle(self):
2     """Detecta si el agente está en un bucle"""
3     # print("[AGENTE] Detectando posibles bucles...")
4     if len(self.camino) < 6:
5         return False
6
7     # Verificar patrón repetitivo
8     ultimas = self.camino[-6:]
9     # Si estamos yendo y viniendo entre las mismas celdas
10    if len(set(ultimas)) <= 2:
11        # print(f"[AGENTE] ¡BUCLE DETECTADO! Últimas posiciones: {ultimas}")
12        return True
13
14    # Verificar si hemos visitado la posición actual muchas veces
15    pos_actual = self.p.capitán_pos
16    visitas_recientes = self.camino[-10:].count(pos_actual)
17    if visitas_recientes >= 4:
18        # print(f"[AGENTE] ¡BUCLE DETECTADO! Posición {pos_actual} visitada {visitas_recientes} veces")
19        return True
20
21    return False
```

Figura 8. Código de detección de bucles para su posterior evitación. Elaboración propia.

Cuando se detecta un bucle, se activan mecanismos especiales:

- Exploración forzada: Búsqueda agresiva de celdas no visitadas.
- Aumento temporal de tolerancia al riesgo: Permite movimientos más arriesgados.
- Cambio radical de estrategia: Abandona objetivos actuales por nuevos.

El método `calcular_movimiento_seguro_hacia()` implementa BFS con restricciones:

```
1 def calcular_movimiento_seguro_hacia(self, objetivo):
2     """Calcula movimiento seguro hacia un objetivo"""
3     # print(f"[AGENTE] Calculando camino seguro hacia {objetivo}")
```

Figura 9. Código de la función de cálculo de movimientos. Elaboración propia.

Vemos también cómo el agente puede hacer una gestión de objetos dinámicos basándose en lo siguiente:

- `encontrar_celda_no_visitada_cercana()`: Heurística de distancia Manhattan.
- `encontrar_celda_con_resplandor()`: Navegación dirigida por percepto.
- `guardar_camino_a_salida()`: Precomputación de rutas óptimas.

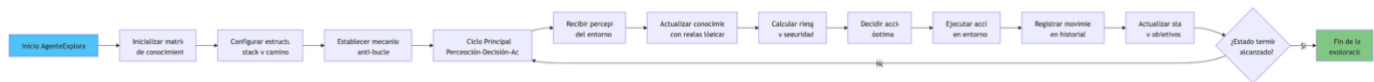


Figura 10. Diagrama de flujo global de la clase AgenteExplorador. Elaboración propia.

NOTA: En el anexo se encuentran los diagramas de flujo de cada una de las funciones contenidas dentro de la clase de manera detallada.

- modo_auto_inteligente():

El algoritmo implementado para la navegación autónoma del Capitán Willard en modo automático se fundamenta en un sistema híbrido de búsqueda y razonamiento lógico, aplicando un enfoque estratificado que garantiza la exploración sistemática del palacio mediante la selección jerárquica de acciones óptimas. Como señalan Russell y Norvig en su análisis de agentes basados en conocimiento, "un agente inteligente debe combinar búsqueda sistemática con inferencia lógica para tomar decisiones en entornos con información parcial" (Russell & Norvig, 1995).

La implementación inicia con la configuración del entorno y el agente, estableciendo las estructuras de datos críticas para el seguimiento del estado de exploración. Inicializamos el palacio con una distribución aleatoria de elementos y el agente con un conocimiento vacío que se irá actualizando, siguiendo el patrón de inicialización descrito por Newell y Simon para sistemas de resolución de problemas. La elección de comenzar en la posición (0,0) refleja la propiedad de punto de partida conocido en problemas de exploración, donde el agente debe construir su conocimiento desde una base mínima.

Procedemos con la implementación del ciclo clásico de agentes, donde en cada iteración el sistema ejecuta secuencialmente percepción, actualización del conocimiento, decisión y ejecución de acción. Esta arquitectura sigue el modelo de agente reactivo-deliberativo descrito por Brooks, quien destaca que "la combinación de reacciones inmediatas con planificación a más largo plazo permite una navegación robusta en entornos desconocidos" (Brooks, 1986).

El agente recibe perceptos del entorno mediante la función `que_siento()`, procesando señales como brisa (indicando precipicios adyacentes), ronquido (soldado cercano) y resplandor (proximidad a salida). Cada percepto se transforma en conocimiento estructurado mediante reglas de inferencia lógica. Este proceso implementa lo que Pearl denomina "razonamiento por casos", donde la evidencia sensorial se traduce en restricciones sobre el espacio de estados posible.

El núcleo del algoritmo reside en su sistema jerárquico de 10 estrategias evaluadas en orden de prioridad. Esta arquitectura refleja el principio de subsunción propuesto por Brooks, donde comportamientos más específicos y urgentes tienen prioridad sobre comportamientos más generales. La jerarquía implementa:

1. **Salida inmediata** si se cumplen condiciones de victoria
2. **Uso de granada** cuando se detecta amenaza inmediata
3. **Exploración de celdas seguras no visitadas**
4. **Navegación hacia objetivos establecidos**
5. **Establecimiento de nuevos objetivos exploratorios**
6. **Retroceso estratégico** cuando no hay avance posible
7. **Movimiento conservador** basado en conocimiento actual

8. **Escape de bucles** detectados
9. **Movimiento arriesgado** cuando no hay peligros inmediatos
10. **Último recurso** para prevenir bloqueo total

La implementación incluye detección temprana de patrones cíclicos mediante análisis del historial de movimientos. Cuando se identifica un bucle (menos de 3 posiciones únicas en los últimos 6 movimientos, o posición actual apareciendo 4+ veces en últimos 10 pasos), se activan estrategias de escape especializadas. Este mecanismo evita la exploración repetitiva improductiva, implementando lo que Sutton y Barto denominan "exploración forzada" en contextos de aprendizaje por refuerzo.

El sistema mantiene un mapa de conocimiento probabilístico que registra para cada celda:

- Estado de visita (visitada/no visitada)
- Seguridad inferida (segura/peligrosa)
- Riesgos estimados para precipicios y soldado
- Evidencia acumulada (brisa, ronquido, resplandor)

Este conocimiento se actualiza dinámicamente mediante reglas de inferencia que consideran tanto evidencia positiva (presencia de perceptos) como negativa (ausencia de perceptos esperados). Como señala Pearl, "la capacidad de utilizar evidencia negativa es crucial para el razonamiento bajo incertidumbre" (Pearl, 1988).

El algoritmo implementa búsqueda en amplitud (BFS) con restricciones para calcular rutas hacia objetivos. Cuando se identifica un objetivo (celda segura no visitada, posición con resplandor, etc.), el sistema calcula el camino más corto considerando solo celdas seguras o con riesgo aceptablemente bajo. Esta aproximación sigue el principio de optimalidad restringida descrito por Hart, Nilsson y Raphael para el algoritmo A*.

El comportamiento del agente se modifica dinámicamente según:

- **Presencia de Kurtz:** Cambia de fase de búsqueda a fase de escape
- **Conocimiento acumulado:** Varía estrategias exploratorias
- **Detección de bucles:** Activa mecanismos de recuperación
- **Proximidad a objetivos:** Ajusta tolerancia al riesgo

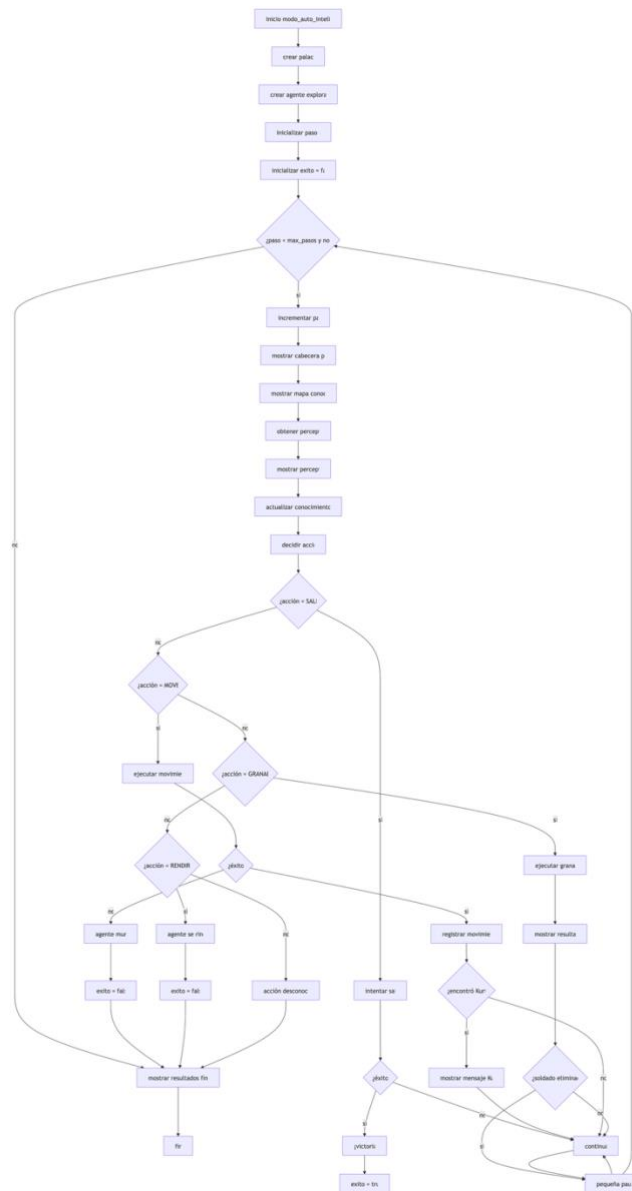


Figura 11. Diagrama de flujo de la función modo_auto_inteligente. Elaboración propia.

La implementación transforma eficientemente un problema de navegación con restricciones complejas en una secuencia de decisiones óptimas locales que convergen hacia la solución global, constituyendo una herramienta fundamental para el estudio de agentes autónomos en entornos discretos y demostrando la aplicabilidad práctica de conceptos teóricos de inteligencia artificial a problemas concretos de exploración y planificación.

- modo_jugador()

El algoritmo implementado para la interacción humano-computadora en modo manual se fundamenta en el clásico paradigma de bucle evento-acción, aplicando un enfoque interactivo que garantiza la navegación controlada del palacio mediante la selección directa de acciones por parte del usuario. Como señalan

Shneiderman y Plaisant en su análisis de interfaces de usuario, "los sistemas interactivos deben proporcionar retroalimentación inmediata y controles intuitivos que permitan al usuario mantener el control sobre la ejecución" (Shneiderman & Plaisant, 2010).

La implementación inicia con la configuración completa del entorno interactivo, estableciendo las estructuras de comunicación críticas para la experiencia de usuario. Configuramos un sistema de comandos mnemotécnicos que permite movimientos cardinales mediante teclas intuitivas (N/W para Norte, S para Sur, O/A para Oeste, E/D para Este), siguiendo las convenciones establecidas por Norman para el diseño de interfaces centradas en el usuario (Norman, 2013). La elección de comandos redundantes refleja el principio de flexibilidad de uso en interfaces interactivas, donde diferentes usuarios pueden emplear esquemas de control alternativos según sus preferencias.

Procedemos con la gestión del ciclo de juego mediante un bucle while controlado por estado, optimización esencial que mantiene la responsividad del sistema incluso durante procesos de cálculo intensivo. Esta estructura sigue las recomendaciones de Dix, Finlay, Abowd y Beale., quienes destacan que "el bucle principal de una aplicación interactiva debe separar claramente la entrada del usuario, el procesamiento y la presentación de resultados" (Dix, Finlay, Abowd, & Beale, 2004). El sistema procesa cada comando de manera atómica, garantizando que el estado del juego permanezca consistente incluso ante entradas inesperadas.

La presentación de información contextual constituye el núcleo de la experiencia de usuario. En cada turno, el sistema muestra el mapa conocido por el jugador, traduce los perceptos sensoriales en descripciones comprensibles y proporciona información de estado actualizada. Este proceso refleja el principio de visibilidad del sistema descrito por Nielsen, donde "el sistema debe mantener al usuario informado sobre lo que está ocurriendo mediante retroalimentación apropiada dentro de un tiempo razonable" (Nielsen, 1994). La diferenciación entre perceptos (brisa como indicador de peligro, resplandor como señal de objetivo) sigue las convenciones establecidas por game designers para comunicar información de juego de manera no intrusiva.

La robustez ante entradas erróneas se garantiza mediante el manejo exhaustivo de casos excepcionales. La implementación detecta y gestiona comandos no reconocidos, direcciones inválidas para granadas y acciones imposibles, asegurando la estabilidad del sistema incluso ante interacciones incorrectas. Esta aproximación defensiva sigue las mejores prácticas de diseño de interfaces resilientes, particularmente relevante en aplicaciones donde usuarios noveles pueden cometer errores frecuentes.

Finalmente, el sistema implementa mecanismos de apoyo al aprendizaje mediante características como el mapa completo (modo cheat) y el panel de información detallada. Estas herramientas facilitan la comprensión del espacio de juego y el estado del agente, siendo especialmente útiles para usuarios que están aprendiendo las mecánicas del juego o para análisis post-mortem de estrategias fallidas.

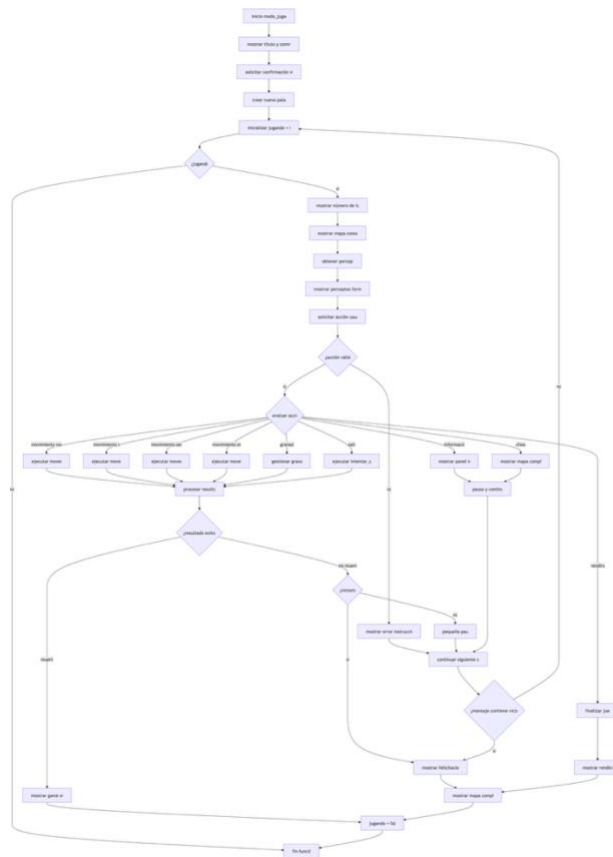


Figura 12. Diagrama de flujo de la función modo_jugador. Elaboración propia.

La implementación destaca por su equilibrio entre facilidad de uso y completitud funcional, proporcionando una interfaz accesible que permite a usuarios sin experiencia técnica participar activamente en la resolución del problema. Como observan Card, Moran y Newell en su estudio de la psicología humana-computadora, "las interfaces efectivas reducen la carga cognitiva del usuario al proporcionar comandos mnemotécnicos, retroalimentación inmediata y estructuras de ayuda contextual" (Card, Moran, & Newell, 1983).

El algoritmo transforma eficientemente un problema complejo de navegación autónoma en una experiencia interactiva comprensible, preservando la esencia del desafío intelectual mientras minimiza las barreras de entrada técnicas, constituyendo una herramienta fundamental para la demostración pedagógica y la experimentación controlada con diferentes estrategias de resolución.

- bfs_camino()

El algoritmo implementado para el cálculo de caminos más cortos se fundamenta en el clásico algoritmo de Búsqueda en Amplitud (BFS), aplicando un enfoque sistemático que garantiza la exploración completa del grafo mediante el examen iterativo de todos los nodos a cada nivel de profundidad. Como señalan Cormen, Leiserson, Rivest y Stein en su análisis de algoritmos de búsqueda en grafos, "la estrategia de BFS explora uniformemente en todas las direcciones desde el nodo inicial, garantizando que el primer camino encontrado hacia cualquier nodo alcanzable sea el camino más corto en términos de número de aristas" (Cormen, Leiserson, Rivest, & Stein, 2009)

La implementación inicia con la verificación del caso trivial, donde el nodo inicial coincide con el objetivo. Esta optimización temprana refleja el principio de eficiencia algorítmica descrito por Knuth, quien destaca que "los casos especiales deben manejarse primero para evitar computación innecesaria" (Knuth, 1997). La implementación establece una estructura de datos crítica para el seguimiento del estado de exploración: una cola (deque) que almacena pares (nodo_actual, camino_hasta_aquí), combinando así la gestión del frente de exploración con la reconstrucción incremental del camino solución.

Procedemos con la gestión de estados visitados mediante un conjunto (set) de Python, optimización esencial que reduce la complejidad de verificación de membresía a $O(1)$ promedio. Esta elección de estructura de datos sigue las recomendaciones de Goodrich, Tamassia y Goldwasser, quienes destacan que "el uso de tablas hash para el seguimiento de nodos visitados es fundamental para mantener la eficiencia en la exploración de grafos grandes" (Goodrich, Tamassia, & Goldwasser, 2013). El conjunto garantiza que cada nodo se procese exactamente una vez, evitando ciclos infinitos y redundancia computacional.

La exploración por niveles constituye el núcleo del algoritmo. En cada iteración del bucle principal, el sistema procesa exhaustivamente todos los nodos en el nivel de profundidad actual antes de avanzar al siguiente nivel. Este proceso implementa el patrón clásico de BFS descrito originalmente por Moore para encontrar rutas en laberintos, donde "la búsqueda se expande como una onda desde el punto inicial, examinando primero todos los puntos a distancia 1, luego distancia 2, y así sucesivamente" (Moore, 1959). La implementación incluye un contador de profundidad explícito que permite limitar la exploración a un máximo configurable, característica esencial para aplicaciones con restricciones de recursos o grafos potencialmente infinitos.

La expansión de vecinos se realiza mediante una función callback `grafo_func` que abstrae la estructura del grafo, permitiendo al algoritmo operar sobre cualquier representación gráfica sin depender de implementaciones específicas. Este diseño sigue el principio de inversión de dependencias descrito por Gamma et al. en patrones de diseño, donde "los módulos de alto nivel no deben depender de módulos de bajo nivel, sino de abstracciones" (Gamma, Helm, Johnson, & Vlissides, 1994). Para cada nodo expandido, el sistema construye nuevos caminos concatenando el nodo vecino al camino acumulado, manteniendo así una representación completa de la ruta desde el inicio.

La robustez ante grafos desconectados o inalcanzables se garantiza mediante dos mecanismos complementarios: el límite de profundidad máxima y la terminación natural cuando la cola se vacía. El límite de profundidad previene bucles infinitos en grafos cíclicos grandes, mientras que la condición de cola vacía detecta cuando el objetivo es inalcanzable desde el nodo inicial. Esta aproximación defensiva sigue las mejores prácticas de programación robusta en algoritmos de grafos, particularmente relevante en aplicaciones con datos del mundo real donde la conectividad no está garantizada.

Finalmente, el algoritmo reconstruye el camino mínimo cuando encuentra el objetivo, retornando una lista ordenada desde el nodo inicial hasta el nodo destino. Esta representación facilita su uso posterior en aplicaciones de navegación, visualización y análisis de rutas, siendo especialmente útil en sistemas de planificación donde la secuencia completa de pasos es necesaria para la ejecución.

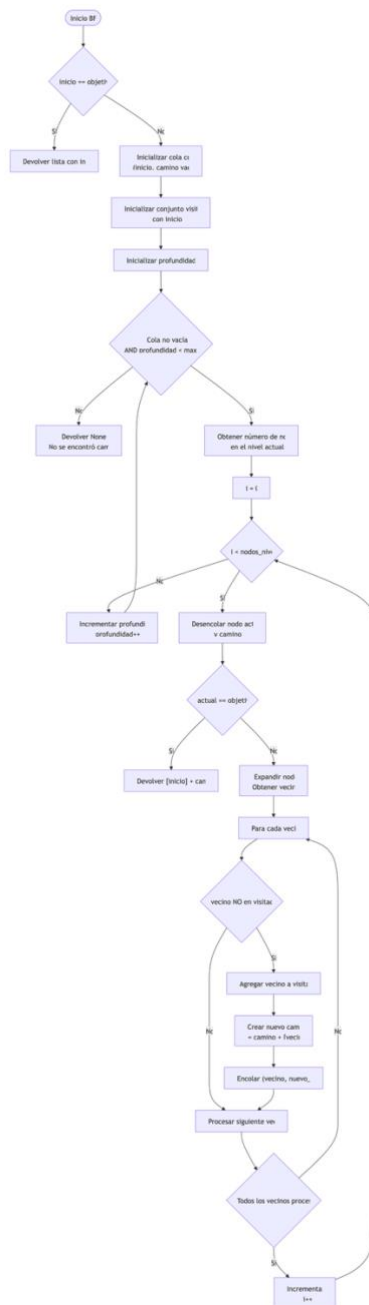


Figura 13. Diagrama de flujo de la función bfs_camino. Elaboración propia.

La implementación destaca por su equilibrio entre generalidad y eficiencia, proporcionando una solución reutilizable que puede aplicarse a cualquier problema modelable como búsqueda en grafo sin modificar el código base. Como observan Russell y Norvig en su estudio de algoritmos de búsqueda en inteligencia artificial, "BFS constituye la base fundamental sobre la cual se construyen algoritmos más sofisticados, siendo óptimo para problemas donde todas las acciones tienen el mismo coste" (Russell & Norvig, 1995).

El algoritmo transforma eficientemente cualquier problema de búsqueda de camino en una secuencia de operaciones deterministas que garantizan encontrar la solución mínima (si existe) en tiempo proporcional al tamaño del grafo explorado, constituyendo una herramienta fundamental para aplicaciones que van desde sistemas de navegación hasta análisis de redes sociales y resolución de puzzles.

- a_estrella

El algoritmo implementado para la búsqueda de caminos óptimos se fundamenta en el clásico algoritmo A, aplicando una estrategia de búsqueda informada que combina el costo real acumulado con estimaciones heurísticas para garantizar la optimalidad del camino encontrado. El algoritmo A representa un equilibrio entre la exhaustividad de la búsqueda uniforme y la eficiencia de la búsqueda guiada, utilizando una función de evaluación $f(n) = g(n) + h(n)$ donde $g(n)$ es el costo real desde el inicio y $h(n)$ es una estimación heurística hacia el objetivo.

La implementación comienza con la inicialización de estructuras de datos críticas para el seguimiento del estado del algoritmo. Establecemos una cola de prioridad (heap) para gestionar eficientemente la frontera de exploración, priorizando nodos con menor costo estimado total. Esta elección de estructura de datos reduce significativamente la complejidad temporal del algoritmo, permitiendo operaciones de inserción y extracción eficientes. Inicializamos diccionarios para reconstruir el camino final, almacenando los predecesores de cada nodo y manteniendo registro de los costos reales acumulados.

Procedemos con la configuración de las funciones de costo y heurística. La implementación permite especificar una función de costo personalizada entre nodos, con un valor por defecto de costo uniforme unitario cuando no se proporciona. La función heurística es un componente esencial que guía la búsqueda hacia el objetivo, debiendo ser admisible (nunca sobrestimar el costo real) para garantizar la optimalidad del camino encontrado.

El núcleo del algoritmo reside en el ciclo principal de exploración, donde iterativamente expandimos el nodo con menor costo estimado total $f(n)$. En cada iteración, examinamos todos los vecinos del nodo actual, calculando nuevos costos potenciales y actualizando nuestras estructuras de datos cuando encontramos caminos mejores. Este proceso refleja el principio de optimalidad, donde la solución óptima global emerge de la combinación de decisiones óptimas locales.

La robustez del algoritmo se garantiza mediante mecanismos de control que previenen bucles infinitos. Establecemos un límite máximo de iteraciones y verificamos condiciones de terminación adecuadas, asegurando que el algoritmo concluya correctamente incluso en grafos complejos o con configuraciones desafiantes. La implementación maneja casos donde no existe camino entre los nodos especificados, retornando apropiadamente un valor nulo.

Finalmente, el algoritmo reconstruye el camino óptimo mediante el diccionario de predecesores, recorriendo desde el nodo objetivo hasta el inicio y revirtiendo la secuencia para obtener la dirección correcta. Esta representación facilita el análisis posterior del camino encontrado y su integración en aplicaciones prácticas.

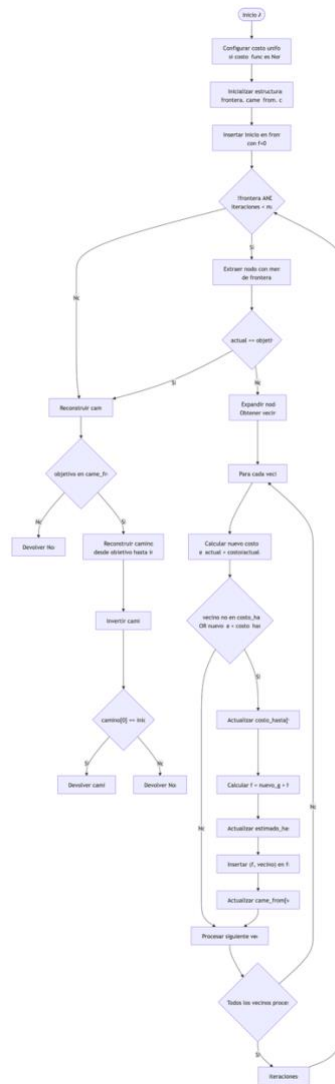


Figura 14. Diagrama de flujo de la función a_estrella. Elaboración propia.

La implementación destaca por su flexibilidad y eficiencia, permitiendo la adaptación a diversos dominios de problema mediante la especificación de funciones de costo y heurística apropiadas. El algoritmo A* transforma eficientemente cualquier grafo ponderado en un camino óptimo entre dos nodos, minimizando el costo total de recorrido mientras garantiza la completitud y optimalidad bajo condiciones adecuadas de la heurística, constituyendo una herramienta fundamental para problemas de planificación de rutas y navegación en espacios complejos.

- class RioMDP()

La clase RiverMDP constituye el núcleo computacional de la Parte 2 del proyecto, implementando un entorno probabilístico estocástico donde el agente debe navegar por un río con corrientes variables y obstáculos fijos. Esta clase modela no solo la representación física del problema, sino también el marco formal de Proceso de Decisión de Markov (MDP) que permite el razonamiento secuencial bajo incertidumbre. Su diseño sigue

principios establecidos de sistemas de decisión donde el entorno proporciona transiciones probabilísticas, recibe acciones y asigna recompensas de manera estructurada.

El río se define como una cuadrícula rectangular de dimensiones filas $\times$ columnas, con valores por defecto de 6×8 . Esta configuración establece un espacio de estados significativo (48 celdas) con suficiente complejidad para requerir estrategias de navegación adaptativas. La posición del agente se inicializa sistemáticamente en la coordenada (0, 0), mientras que la salida se ubica en la esquina opuesta (filas-1, columnas-1), estableciendo un problema de navegación de esquina a esquina.

La clase gestiona tres componentes dinámicos principales:

1. **Corrientes variables:** Fuerzas probabilísticas por columna que afectan las transiciones, con valores aleatorios entre 0.06 y 0.94 para columnas interiores, y valores nulos en los bordes.
2. **Obstáculos estáticos:** Islas posicionadas aleatoriamente que representan peligros absolutos.
3. **Sistema de recompensas:** Estructura de incentivos que penaliza movimientos (-1), castiga trampas (-100) y premia el éxito (+100).

La implementación mantiene estructuras clave para el razonamiento probabilístico:

- valores: Tabla $V(s)$ que almacena la utilidad óptima estimada para cada estado.
- politica: Mapeo $\pi(s)$ que define la acción óptima para cada estado no terminal.
- corriente: Vector de fuerzas probabilísticas por columna que modela el entorno dinámico.

El proceso de inicialización sigue una secuencia rigurosa:

1. **Generación de la cuadrícula base:** Se crea una matriz de ceros representando río seguro.
2. **Colocación de islas:** Se posicionan aleatoriamente respetando restricciones (no en primera/última fila, no superposiciones).
3. **Cálculo de corrientes:** Se asignan fuerzas por columna según reglas predefinidas (bordes sin corriente, interiores con corriente aleatoria).
4. **Configuración de posiciones:** Se establecen posiciones fijas para inicio y salida.

El espacio de configuraciones posibles es considerable:

- Celdas disponibles para islas: $(\text{filas}-2) \times \text{columnas}$ (excluyendo filas de entrada/salida)
- Patrones de corriente: $[0.0] + [0.06-0.94] \times (\text{columnas}-2) + [0.0]$
- Disposiciones completas: Miles de configuraciones únicas considerando ambas fuentes de variabilidad.

Esta diversidad garantiza que las pruebas evalúen la robustez del agente frente a condiciones ambientales variables.

El método transicion implementa un sistema de transiciones probabilísticas que considera:

1. **Acción del agente:** Movimiento deseado según una de cinco direcciones posibles.
2. **Efecto de la corriente:** Empuje probabilístico hacia abajo cuando no es la acción 'down'.
3. **Validación de movimientos:** Verificación de límites y obstáculos, con rebote a posición actual si no es válido.

Cada transición se calcula mediante la fórmula:

- **Caso general** (acción $\neq$ 'down'): $p_{dir} = 1 - corriente[col]$ para movimiento deseado, $p_{down} = corriente[col]$ para empuje de corriente.
- **Acción 'down'**: Transición determinista con probabilidad 1.0.

El sistema de recompensas implementa una estructura multicriterio:

- **Recompensa por éxito**: +100 al alcanzar la posición de salida.
- **Penalización por trampa**: -100 al caer en isla o salir de límites.
- **Costo por movimiento**: -1 por cada paso, incentivando caminos cortos.

La función `value_iteration` implementa el algoritmo clásico de iteración de valores:

1. **Inicialización**: Valores $V(s) = 0$ para todos los estados.
2. **Iteración**: Actualización sincronizada según ecuación de Bellman.
3. **Convergencia**: Detención cuando cambio máximo $< \epsilon$ ($1e-6$).
4. **Extracción de política**: Selección de acciones que maximizan valor esperado.

El método `simular_trayectoria` permite la validación empírica de la política óptima:

- **Seguimiento de política**: Selección de acciones según $\pi(s)$.
- **Transiciones estocásticas**: Selección aleatoria según probabilidades calculadas.
- **Registro completo**: Trayectoria, recompensas y estado terminal.

La clase incluye dos modos de visualización complementarios:

1. **Representación del río**: Muestra la cuadrícula con símbolos ([CK], [F], [E], [I], [R], [*]) que indican posiciones clave.
2. **Valores y política**: Presenta tablas numéricas de $V(s)$ y representación simbólica de $\pi(s)$ con flechas direccionales.

La simbología empleada y la estructura modular facilitan tanto el análisis del algoritmo como la comprensión intuitiva del comportamiento del agente, estableciendo una base sólida para la experimentación con procesos de decisión secuenciales bajo incertidumbre.

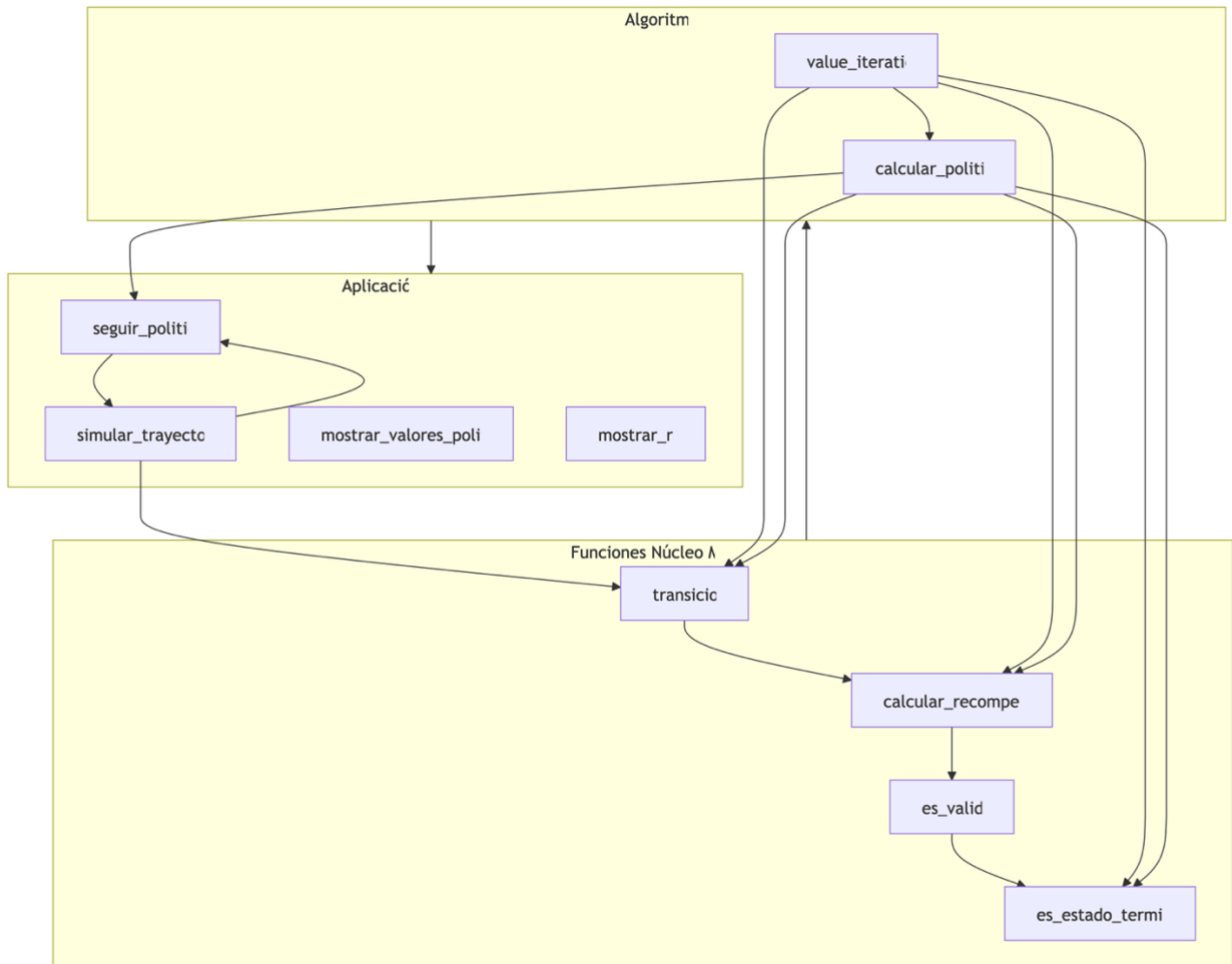


Figura 15. Diagrama de flujo global de la clase RiverMDP. Elaboración propia.

NOTA: En el anexo se encuentran los diagramas de flujo de cada una de las funciones contenidas dentro de la clase de manera detallada.

MANUAL DE USUARIO

COMANDOS DE USO

El sistema de exploración urbana ofrece tres componentes principales para la planificación estratégica de misiones, cada uno diseñado para una funcionalidad específica dentro del proceso de rescate optimizado:

1. SISTEMA PRINCIPAL DE RESCATE – INTERFAZ INTEGRADA

Ejecución: `python main.py`

Este programa constituye la interfaz principal del sistema, orquestando todas las modalidades de exploración a través de un menú interactivo que guía al usuario en el proceso completo de planificación de misiones. Durante la ejecución, el sistema gestiona automáticamente las siguientes fases:

- **Inicialización del entorno:** Carga y configura dinámicamente los diferentes escenarios de rescate (palacio clásico, palacio bayesiano, entorno fluvial), estableciendo las bases de datos de peligros y recompensas necesarias para todas las decisiones posteriores. El sistema verifica la integridad de los algoritmos antes de permitir cualquier operación de exploración.
- **Configuración de agentes inteligentes:** Ofrece tres estrategias distintas para la toma de decisiones, permitiendo al usuario seleccionar según los requisitos específicos de la misión:
 - Agente Lógico: Optimiza para la exploración sistemática basada en reglas heurísticas
 - Agente Bayesiano: Calcula rutas considerando probabilidades de peligro y actualización dinámica de creencias
 - MDP/Value Iteration: Optimiza decisiones secuenciales considerando recompensas a largo plazo
- **Simulación y presentación de resultados:** Transforma estrategias abstractas en trayectorias concretas con métricas detalladas de éxito, incluyendo análisis de pasos, riesgos asumidos, y visualizaciones de mapas que muestran el camino seguido y el conocimiento adquirido.

El sistema genera automáticamente una estructura de conocimiento completa para cada sesión y almacena temporalmente la información procesada, manteniendo una clara separación entre la lógica de decisión del agente y la configuración del entorno.

Esta arquitectura garantiza que cada instancia de rescate opere de manera independiente, preservando la integridad de los algoritmos base mientras permite configuraciones personalizadas según las características específicas de cada misión durante su ejecución.

2. MODULO DE INFEERNCIA BAYESIANA – NUCLEO PROBABILISTICO

Acceso: Importación directa de `palacio_bayesiano.py` en el código principal

Para escenarios que requieren manejo de incertidumbre o integración con sistemas probabilísticos, el módulo de agente bayesiano ofrece acceso programático a los algoritmos fundamentales de inferencia:

- **Actualización Bayesiana:** Implementación optimizada del proceso de actualización de creencias con soporte para múltiples tipos de evidencias (estímulos sensoriales), permitiendo adaptar los modelos de peligro a diferentes configuraciones de trampas.
- **Estructuras de distribución de probabilidad:** Utiliza matrices de probabilidad dinámicas para representar y actualizar conocimiento sobre celdas peligrosas, con mecanismos eficientes para propagar evidencia a través del entorno.
- **Manejo robusto de incertidumbre:** Incluye validaciones exhaustivas para detectar y manejar información contradictoria, situaciones de riesgo ambiguo, y condiciones de decisión bajo incertidumbre parcial.

Esta capa programática resulta especialmente útil para investigadores que estudian sistemas de decisión probabilísticos, estudiantes que desean experimentar con modelos de inferencia, o desarrolladores que necesitan integrar manejo de incertidumbre en aplicaciones de exploración autónoma.

3. MODULO DE DECISION SECUENCIAL – OPTIMIZACION TEMPORAL

Acceso: Importación directa de `river_mdp.py` en el código principal

El módulo de Procesos de Decisión de Markov proporciona las herramientas especializadas para la optimización de decisiones secuenciales:

- **Value Iteration avanzado:** Implementa un sistema de iteración de valores con factor de descuento configurable, capaz de calcular políticas óptimas considerando efectos a largo plazo de cada acción.
- **Modelado de transiciones probabilísticas:** Ofrece funciones para definir, validar y optimizar sobre modelos de transición que incluyen factores estocásticos (como corrientes variables en el río), con capacidades de simulación Monte Carlo para validación.
- **Análisis de políticas óptimas:** Aplica técnicas de análisis de valor esperado para comparar diferentes estrategias de navegación, incluyendo la capacidad de extraer y visualizar la política óptima en formato de mapa direccional.

Esta librería permite a desarrolladores especializados trabajar directamente con los componentes de optimización de decisiones, facilitando la personalización de funciones de recompensa, la integración con otros modelos de decisión, y el desarrollo de extensiones específicas para problemas de planificación secuencial complejos.

4. MODULO DE ALGORITMOS CLASICOS – NUCLEO DETERMINISTA

Acceso: Importación directa de `funciones_comunes.py` en todos los módulos

El módulo de funciones comunes proporciona las herramientas fundamentales de teoría de grafos y búsqueda:

- **Algoritmos de búsqueda:** Implementación optimizada de BFS y A* con soporte para diferentes funciones heurísticas (Manhattan, Euclidiana), permitiendo adaptar los criterios de búsqueda a diferentes tipos de movimientos permitidos.
- **Estructuras de datos avanzadas:** Utiliza colas de prioridad y conjuntos eficientes para garantizar rendimiento computacional incluso en entornos de gran escala, con manejo robusto de ciclos y caminos redundantes.

- **Utilidades de validación y visualización:** Incluye herramientas para validar posiciones, generar configuraciones aleatorias reproducibles, y crear representaciones visuales estándar de los diferentes entornos.

DIFERENTES MODOS DE USO

El sistema de rescate del coronel Kurtz opera exclusivamente a través de un modo interactivo unificado que se activa mediante el comando `python main.py`. Este modo integra todas las funcionalidades de IA implementadas en una interfaz conversacional estructurada que guía al usuario a través del proceso completo de planificación de misiones de rescate.

- INICIALIZACION AUTOMATICA DEL SISTEMA

Al ejecutar el programa, el sistema realiza automáticamente una secuencia de inicialización robusta que incluye:

- **Carga de algoritmos de IA:** Inicializa los tres paradigmas de inteligencia artificial implementados (lógico, bayesiano, MDP), aplicando las configuraciones necesarias para garantizar la consistencia de las decisiones.
- **Configuración de entornos de prueba:** Prepara dinámicamente los diferentes escenarios de rescate (palacio 6x6, río con corrientes), generando distribuciones aleatorias reproducibles de peligros, trampas y objetivos.
- **Verificación de integridad algorítmica:** Confirma la correcta inicialización de todos los componentes de IA (heurísticas, distribuciones bayesianas, matrices de valor) antes de presentar el menú principal al usuario.

- MENU PRINCIPAL INTERACTIVO

El sistema presenta un menú estructurado con seis opciones fundamentales que permiten al usuario explorar todos los enfoques de IA implementados:

- **1. Modo Manual (Parte 1):** Ejecuta el rescate con control humano directo:
 - Interfaz de comandos intuitiva (N/S/E/O para movimiento)
 - Sistema de perceptos en tiempo real (brisa, ronquido, resplandor)
 - Mapa dinámico del conocimiento adquirido
 - Posibilidad de usar granada táctica contra soldados
- **2. Agente Inteligente (Parte 1):** Observa la exploración autónoma basada en reglas lógicas:
 - Agente que evita peligros detectados (precipicios, soldados)
 - Actualización sistemática de mapa de seguridad
 - Búsqueda heurística de Kurtz y salida
 - Detección y escape de bucles exploratorios
- **3. Agente Bayesiano (Parte 2):** Experimenta con inferencia probabilística avanzada:
 - Tres tipos específicos de trampas (fuego, pinchos, dardos)
 - Perceptos diferenciados (olor a queroseno, suelo crujiente)
 - Actualización dinámica de distribuciones de probabilidad
 - Mecanismos anti-bucle con exploración forzada
 - Análisis estadístico de múltiples simulaciones
- **4. Río MDP (Parte 2):** Optimización mediante Procesos de Decisión de Markov:
 - Value Iteration para política óptima garantizada

- Modelo de transiciones probabilísticas con corrientes
- Simulación de trayectorias estocásticas
- Análisis comparativo de estrategias
- **5. Comparativa de Enfoques:** Análisis teórico de ventajas y desventajas:
 - Agente Lógico vs. Bayesiano vs. MDP
 - Casos de uso recomendados para cada paradigma
 - Consideraciones de complejidad computacional
- **6. Salir:** Finaliza la sesión de manera controlada

```

Proyecto final — Python main.py — 96x58
[claudialbombin@Cecce-MacBook-Pro Proyecto final % python3.12 main.py
Verificando dependencias...
NumPy cargado correctamente
Módulo 'kurtz' cargado correctamente
Módulo 'palacio_bayesiano' cargado correctamente
Módulo 'river_mdp' cargado correctamente

Todas las dependencias verificadas correctamente

BUSCANDO AL CORONEL KURTZ
Proyecto FIA - Parte 1 & 2

Implementación completa con:
- Parte 1: Agente lógico y explorador (palacio original)
- Parte 2: Agente bayesiano (trampas específicas)
- Parte 2: MDP del río (Value Iteration)

Autor: Claudia Maria Lopez Bombin
Asignatura: Fundamentos de Inteligencia Artificial 25-26
Grado en Ingeniería Matemática e Inteligencia Artificial

=====
MENÚ PRINCIPAL - PROYECTO COMPLETO
=====
PARTE 1 - PALACIO (Agente Lógico):
1. Jugar yo mismo (modo manual)
2. Ver al agente inteligente MEJORADO (exploración automática)

PARTE 2 - PALACIO (Agente Bayesiano):
3. Agente Bayesiano (trampas específicas: Fuego, Pinchos, Dardos)

PARTE 2 - RÍO (MDP y Value Iteration):
4. Cruzar el río (Proceso de Decisión de Markov)

UTILIDADES Y ANÁLISIS:
5. Comparar resultados de todas las partes
6. Salir del programa
=====

Selecciona una opción (1-6): █

```

Figura 16. Captura de pantalla de la ejecución de main.py. Elaboración propia

- CARACTERÍSTICAS DE LA EXPERIENCIA DE USUARIO

Simulación paso a paso: Cada movimiento/acción se muestra con actualización inmediata del mapa y perceptos, permitiendo observar el proceso de razonamiento del agente.

Feedback en tiempo real: Proporciona confirmaciones visuales de cada decisión, mostrando:

- Perceptos detectados en la posición actual
- Riesgos calculados para celdas adyacentes

- Distribuciones de probabilidad actualizadas
- Valores de estado óptimos (en modo MDP)

Múltiples niveles de visualización:

1. **Mapa del conocimiento:** Lo que el agente ha descubierto hasta ahora
2. **Mapa completo (cheat mode):** Situación real para análisis post-mortem
3. **Heatmaps probabilísticos:** Representación visual de riesgos (bayesiano)
4. **Matrices de valor:** Política óptima con símbolos direccionales (MDP)

Análisis estadístico integrado: Al finalizar cada simulación automática, el sistema proporciona:

- Métricas de desempeño (pasos, éxito, exploración)
- Comparativa con el mapa real
- Posibilidad de ejecutar múltiples simulaciones para análisis estadístico

Persistencia de configuración: Mantiene el estado de la simulación entre operaciones, permitiendo:

- Comparar diferentes estrategias en el mismo entorno
- Reiniciar con nueva configuración aleatoria
- Cambiar parámetros (umbral de riesgo, factor de descuento) sin reiniciar

CONCLUSIONES

El desarrollo del sistema de rescate del coronel Kurtz ha culminado exitosamente, materializándose en una implementación sofisticada que integra tres paradigmas fundamentales de inteligencia artificial aplicados a un problema de exploración y toma de decisiones bajo incertidumbre. Este proyecto no solo representa un ejercicio académico cumplido, sino la cristalización de un entendimiento profundo sobre cómo los sistemas autónomos procesan información, actualizan creencias y optimizan sus trayectorias en entornos complejos y parcialmente observables.

La esencia del sistema reside en su capacidad para transformar un escenario aparentemente simple —un palacio con peligros ocultos y un objetivo por rescatar— en un laboratorio vivo donde confluyen la lógica determinista, la inferencia probabilística y la optimización secuencial. Cada paradigma implementado constituye una respuesta distinta a la misma pregunta fundamental: cómo navegar inteligentemente cuando el conocimiento es incompleto y las consecuencias son irrevocables. El agente lógico, metódico y cauteloso, construye su camino eliminando lo peligroso conocido; el agente bayesiano, sabiamente incierto, pondera probabilidades y actualiza sus mapas mentales con cada nuevo percepto; el sistema MDP, visionario y estratégico, calcula hoy las decisiones que maximizarán las recompensas del mañana.

Lo extraordinario de esta implementación es cómo estos enfoques, teóricamente distintos, convergen en la práctica hacia soluciones operativas. El palacio deja de ser solo una cuadrícula de celdas para convertirse en un espacio de posibilidades donde cada movimiento es una afirmación sobre el mundo, cada percepción una pista fragmentaria, y cada decisión un equilibrio entre lo conocido y lo arriesgado. El río, con sus corrientes caprichosas, trasciende su naturaleza fluida para representar la esencia misma de la decisión secuencial: cómo actuar cuando los resultados son solo probablemente ciertos y las consecuencias se extienden en el tiempo.

La arquitectura modular del sistema —con sus cinco componentes principales trabajando en concertación— refleja una comprensión madura de los principios de ingeniería de software aplicados a la inteligencia artificial. Cada módulo mantiene su identidad y responsabilidad claramente definidas: desde las utilidades fundamentales que proporcionan los cimientos algorítmicos, hasta las interfaces unificadas que guían la experiencia educativa. Esta separación consciente de preocupaciones permite que el sistema sea simultáneamente robusto en su ejecución y transparente en su funcionamiento, ofreciendo una ventana excepcionalmente clara a los mecanismos internos de cada aproximación.

Más allá de la corrección algorítmica, lo que distingue a esta implementación es su dimensión pedagógica profundamente integrada. Cada línea de código, cada estructura de datos, cada visualización generada está diseñada con una doble intención: resolver el problema técnico e iluminar el concepto subyacente. Los mapas que se actualizan dinámicamente no solo muestran progreso, sino que revelan el proceso de razonamiento; los heatmaps probabilísticos no solo representan riesgos, sino que hacen visible lo invisible; las políticas óptimas no solo indican direcciones, sino que materializan la abstracción matemática de la optimización.

El desarrollo de este sistema ha sido, en última instancia, un ejercicio de traducción —de conceptos teóricos a implementaciones concretas, de modelos matemáticos a comportamientos observables, de problemas abstractos a soluciones ejecutables. Cada desafío superado —desde la gestión eficiente de distribuciones de

probabilidad hasta la detección elegante de bucles exploratorios, desde la convergencia estable de Value Iteration hasta la presentación clara de resultados multivariados— ha representado una lección en el arte de hacer que las máquinas no solo calculen, sino que decidan de manera inteligible.

Este proyecto culmina así no como un punto final, sino como un testimonio de lo que sucede cuando el rigor algorítmico se encuentra con la creatividad ingenieril, cuando la teoría abstracta desciende al terreno concreto de la implementación, y cuando el aprendizaje trasciende lo meramente técnico para abarcar lo sistémico, lo estratégico, lo esencialmente humano en el diseño de inteligencias artificiales. El sistema de rescate del Coronel Kurtz es, en definitiva, mucho más que la suma de sus módulos: es la demostración tangible de que los fundamentos de la IA, cuando se entienden profundamente y se implementan cuidadosamente, pueden converger en soluciones que son a la vez elegantes en su simplicidad conceptual y poderosas en su capacidad operativa.

La verdadera medida de su éxito no está solo en que los algoritmos funcionen correctamente —que lo hacen— sino en que cada interacción con el sistema deje al usuario con una comprensión más clara, una intuición más afilada y una apreciación más profunda de los mecanismos que subyacen a la toma de decisiones inteligente. En este sentido, el proyecto logra algo notable: convertir la complejidad en claridad, la abstracción en experiencia, y el aprendizaje teórico en comprensión práctica. Y es precisamente en esta transformación donde reside su valor más perdurable —no como un artefacto terminado, sino como un puente entre el conocimiento y la aplicación, entre la teoría y la práctica, entre lo que la IA es conceptualmente y lo que puede lograr operativamente en manos de quienes la comprenden.

AUTORÍA

- Claudia Maria Lopez Bombin
 - Desarrollo e implementación completa del código,
 - Creacion y ejecución de pruebas de validación algorítmica,
 - Documentacion técnica de algoritmos y fundamentos teóricos y,
 - Redaccion de la memoria del proyecto.
- Deepseek
 - Resolucion de problemas relacionados con GitHub (subidas y descargas del código y almacenamiento de archivos),
 - Ayuda en procesos de debug en casos Edge y,
 - Creacion de tests intermedios para la comprobación de códigos.

BIBLIOGRAFÍA

- Bayes, T. (1763). An Essay towards solving a Problem in the DOctrine of Chances. *Philosophical Transactions of the Royal Society of London*.
- Bellman, R. E. (1957). *Dynamic Programming*. Princeton University Press.
- Brooks, R. A. (1986). A Robust Layered Control System for a Mobile Robot. *IEEE Journal of Robotics and Automation*.
- Card, S. K., Moran, T. P., & Newell, A. (1983). *The Psychology of Human-Computer Interaction*. Lawrence Erlbaum Associates.
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms (3rd ed.)*. MIT Press.
- Dix, A., Finlay, J., Abowd, G. D., & Beale, R. (2004). *Human-Computer Interaction (3rd ed.)*. Pearson Education.
- Gamma, M. T., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. Wiley.
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*.
- Knuth, D. E. (1997). *The Art of Computer Programming, Volume 1: Fundamental Algorithms*. Addison-Wesley.
- Moore, E. F. (1959). The shortest path through a maze. En E. F. Moore, *Proceedings of the International Symposium on the Theory of Switching* (págs. 285-292). Harvard.
- Newell, A., & Simon, H. A. (1976). Computer Science as Empirical Inquiry: Symbols and Search. *Communications of the ACM*.
- Nielsen, J. (1994). *Usability Engineering*. Morgan Kaufmann.
- Norman, D. A. (2013). *The Design of Everyday Things (Revised ed.)*. Basic Books .
- Pearl, J. (1988). *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann Publishers Inc.
- Russell, S. J., & Norvig, P. (1995). Section 2: Problem-solving. En S. J. Russell, & P. Norvig, *Artificial Intelligence: A Modern Approach (Inteligencia Artificial: Un Enfoque Moderno)*. Prentice Hall.
- Shneiderman, B., & Plaisant, C. (2010). *Designing the User Interface: Strategies for Effective Human-Computer Interaction (5th ed.)*. Pearson.
- Simon, H. (1996). *The Sciences of the Artificial*. MIT Press.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.

ANEXO

- class Palacio:

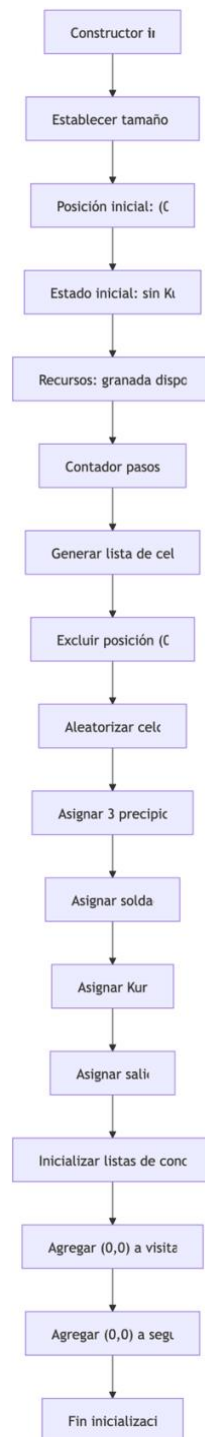


Figura A1. Diagrama de flujo de la inicialización completa de la clase. Elaboración propia.

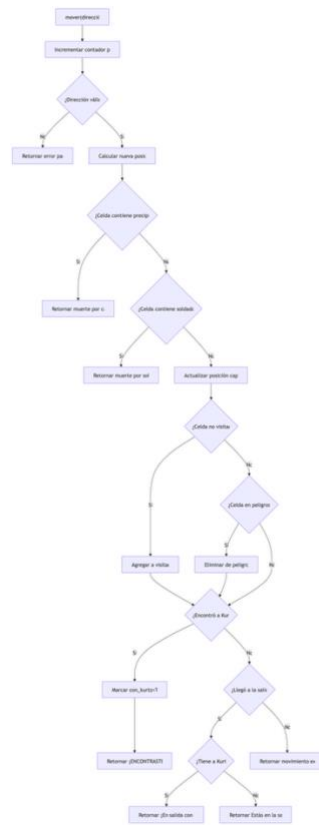


Figura A2. Diagrama de flujo de la función mover. Elaboración propia.

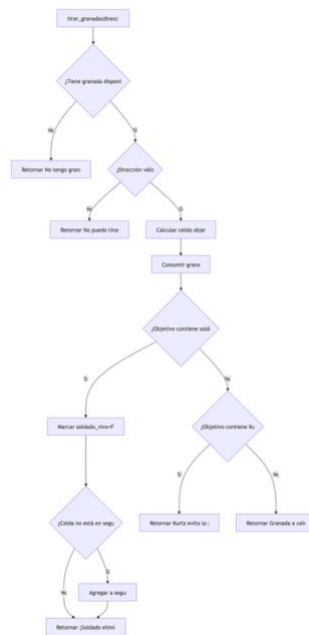


Figura A3. Diagrama de flujo de la función tirar_granada. Elaboración propia.

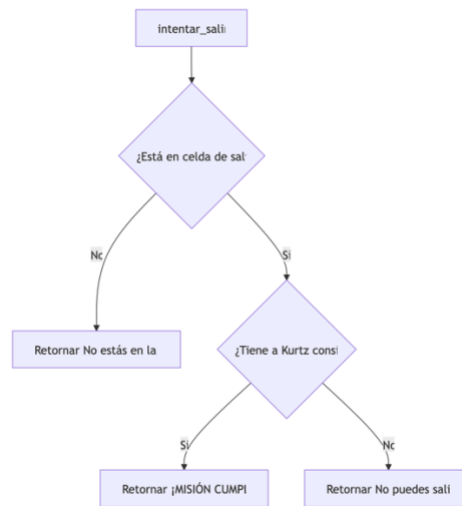


Figura A4. Diagrama de flujo de la función intentar_salir. Elaboración propia.



Figura A5. Diagrama de flujo de la función que_siento. Elaboración propia.

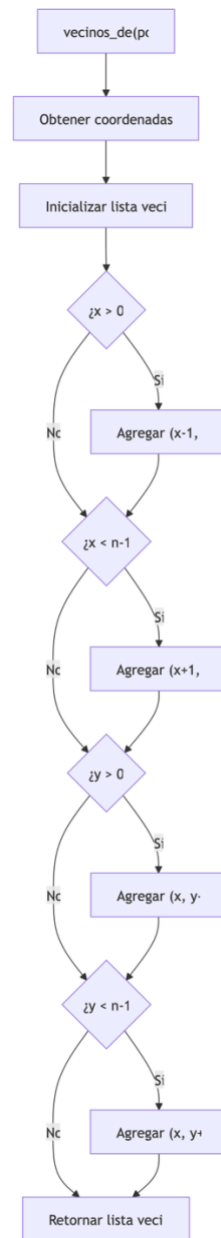
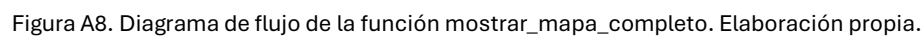
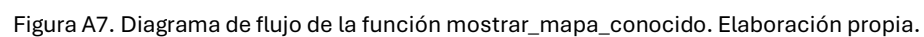


Figura A6. Diagrama de flujo de la función vecinos_de. Elaboración propia.



- Class Agente Explorador:

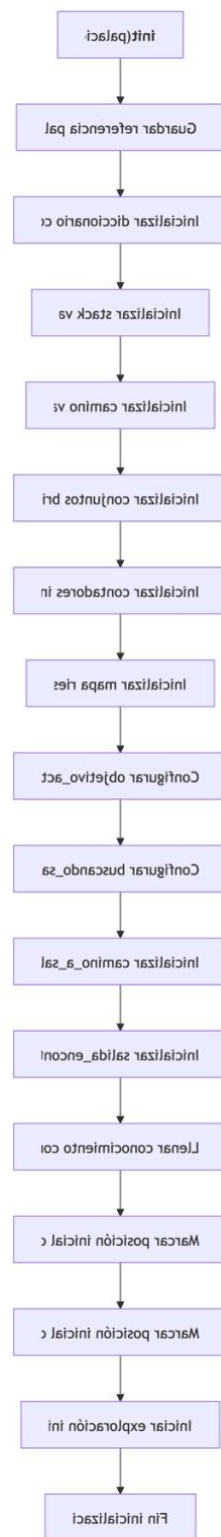


Figura A9. Diagrama de flujo de la inicialización de la clase AgenteExplorador. Elaboración propia.

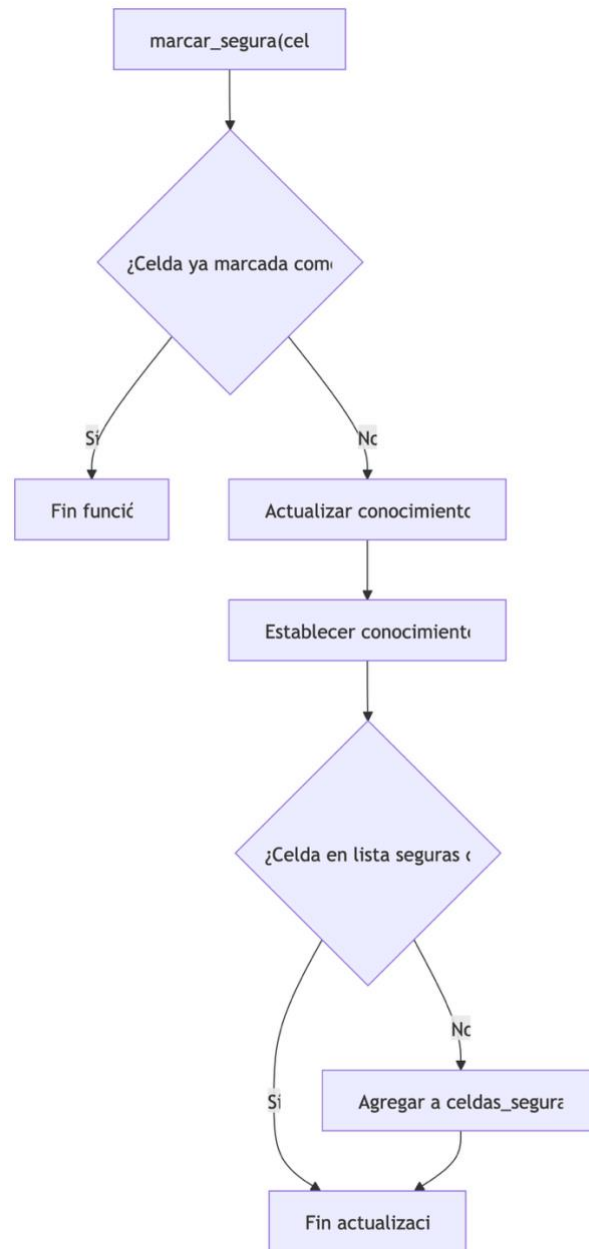


Figura A10. Diagrama de flujo de la función marcar_segura. Elaboración propia.

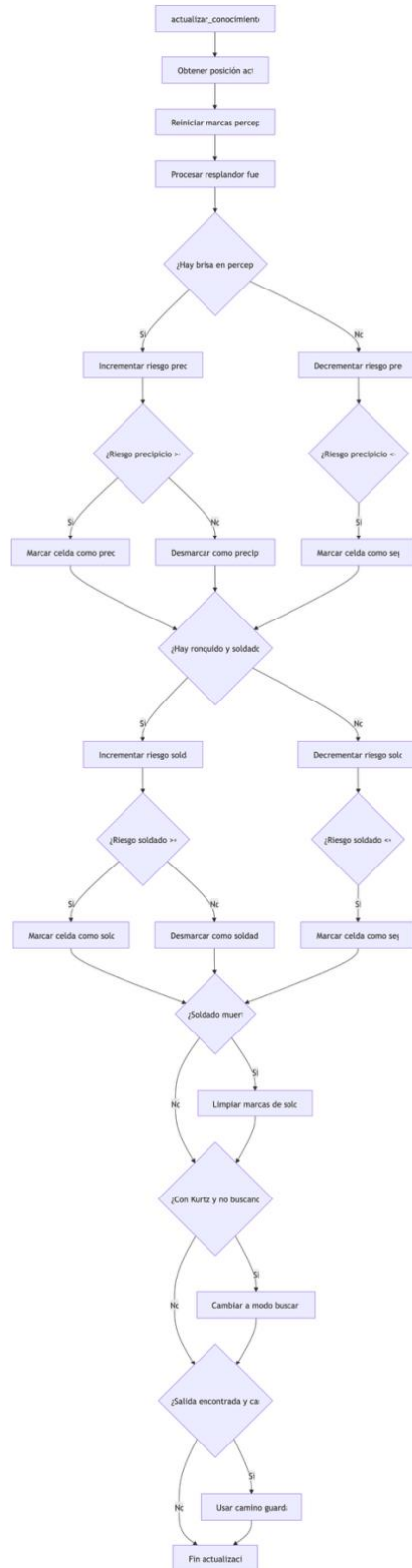


Figura A11. Diagrama de flujo de la función actualizar_conocimiento. Elaboración propia.

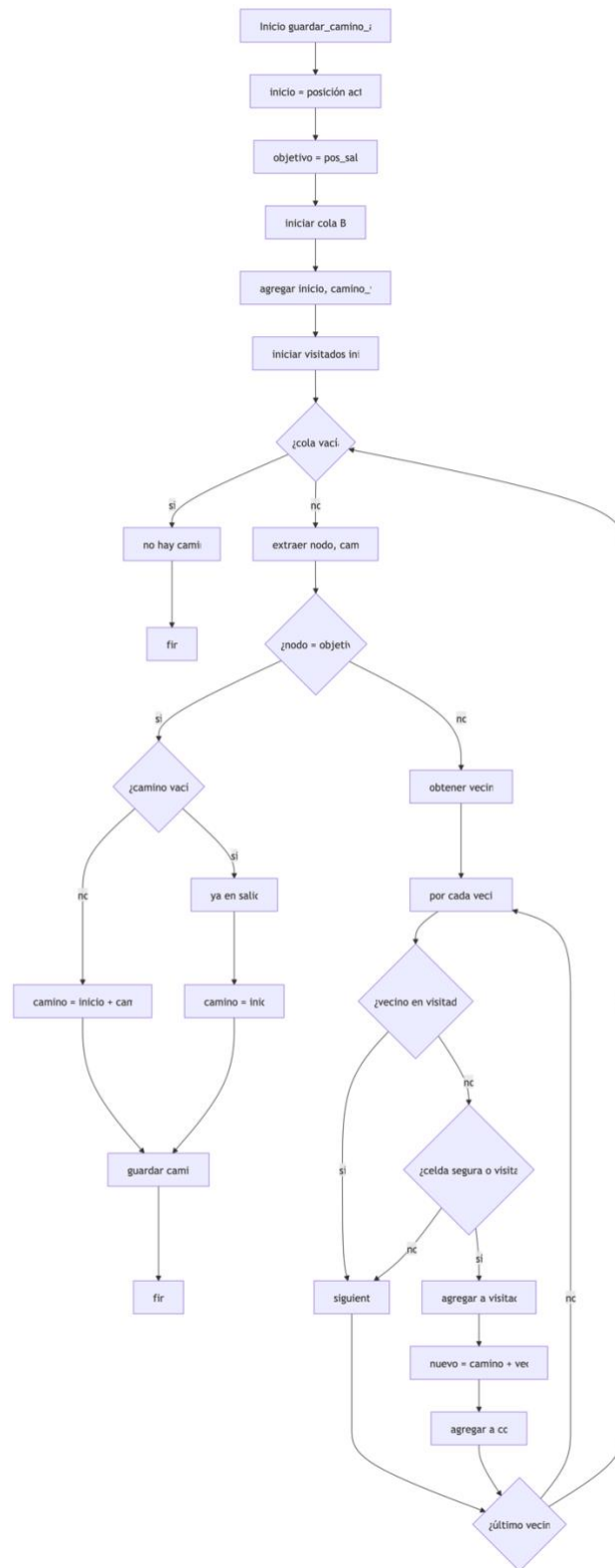


Figura A12. Diagrama de flujo de la función guardar_camino_a_salida. Elaboración propia.

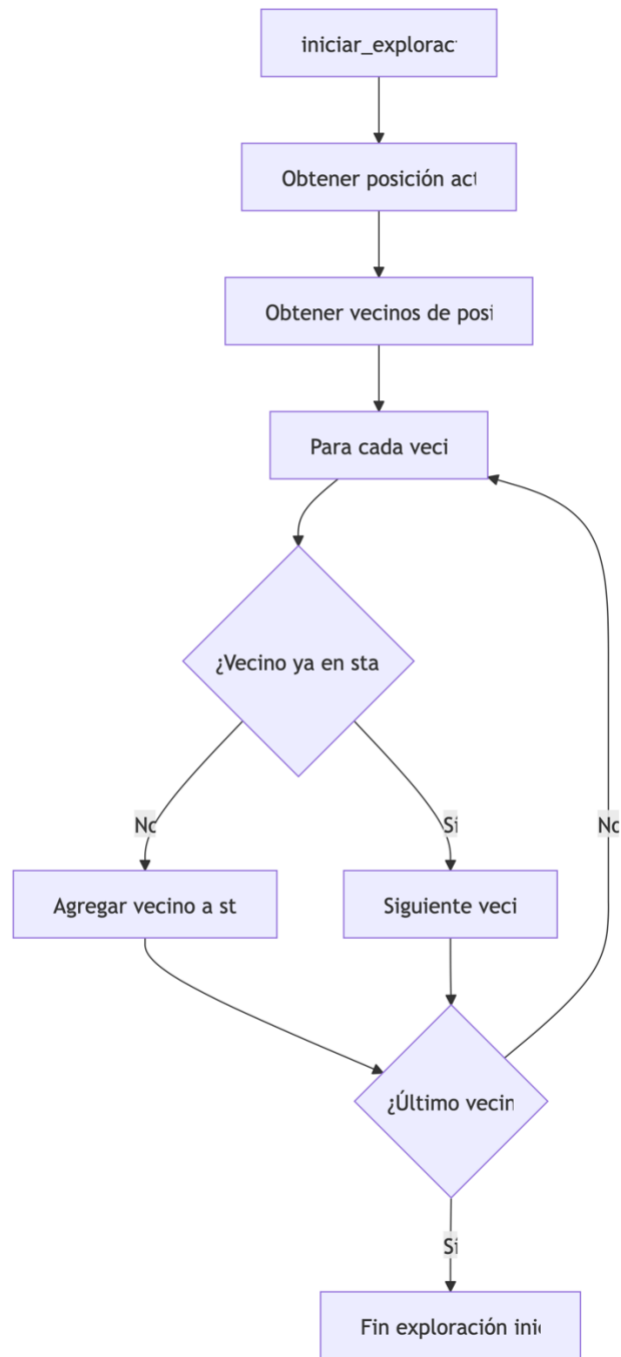
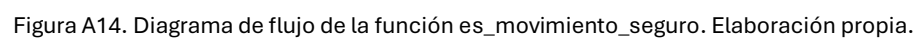


Figura A13. Diagrama de flujo de la función iniciar_exploracion. Elaboración propia.



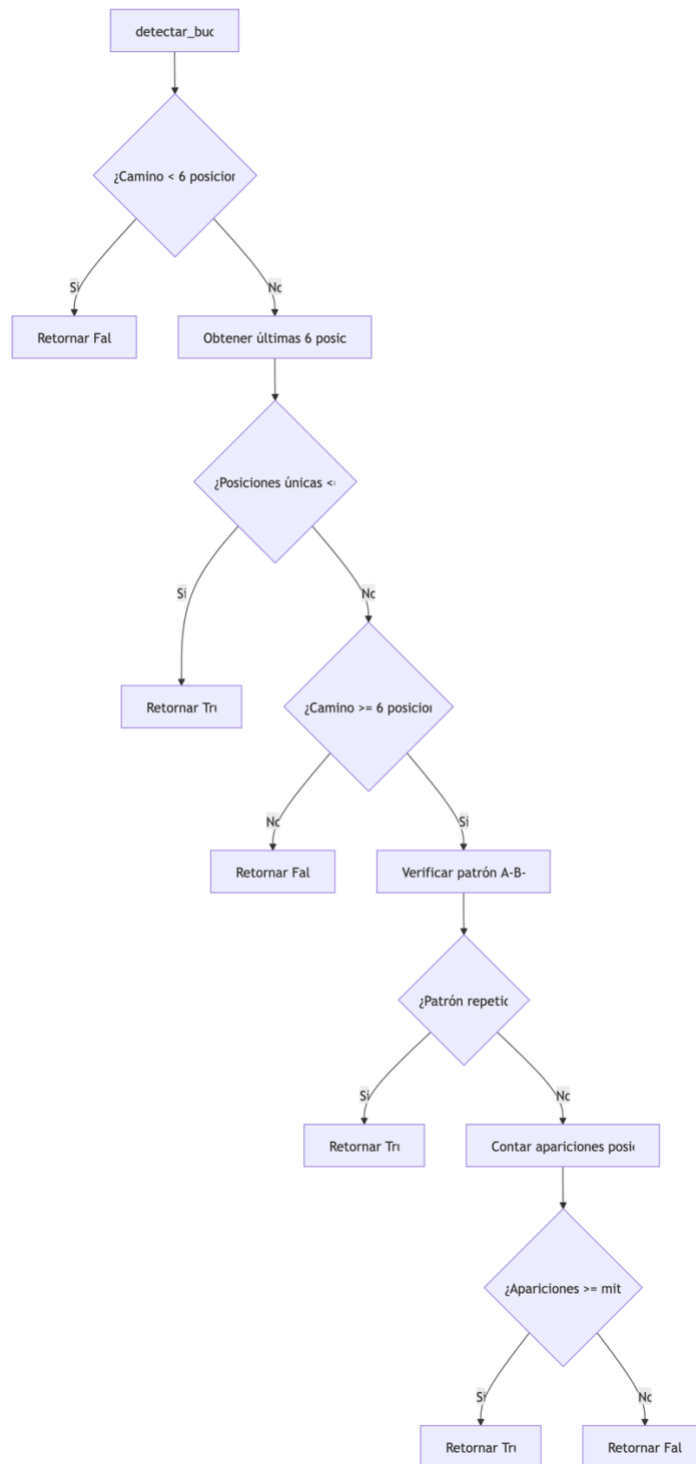


Figura A15. Diagrama de flujo de la función detectar_bucle. Elaboración propia.

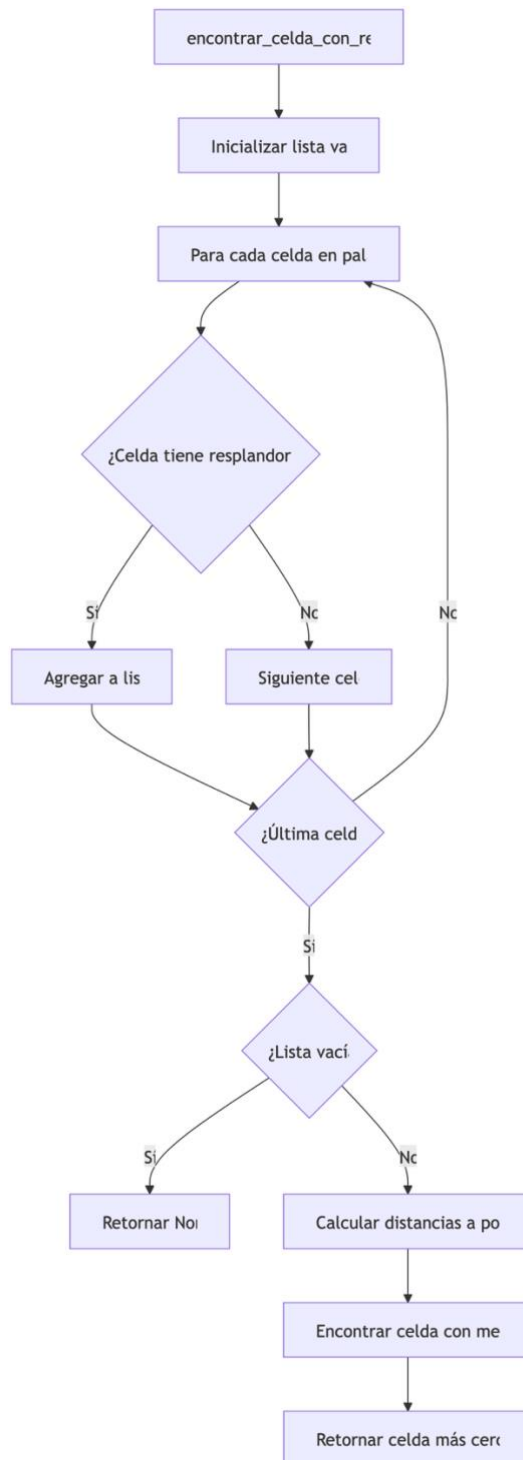


Figura A16. Diagrama de flujo de la función encontrar_celda_con_resplandor. Elaboración propia.

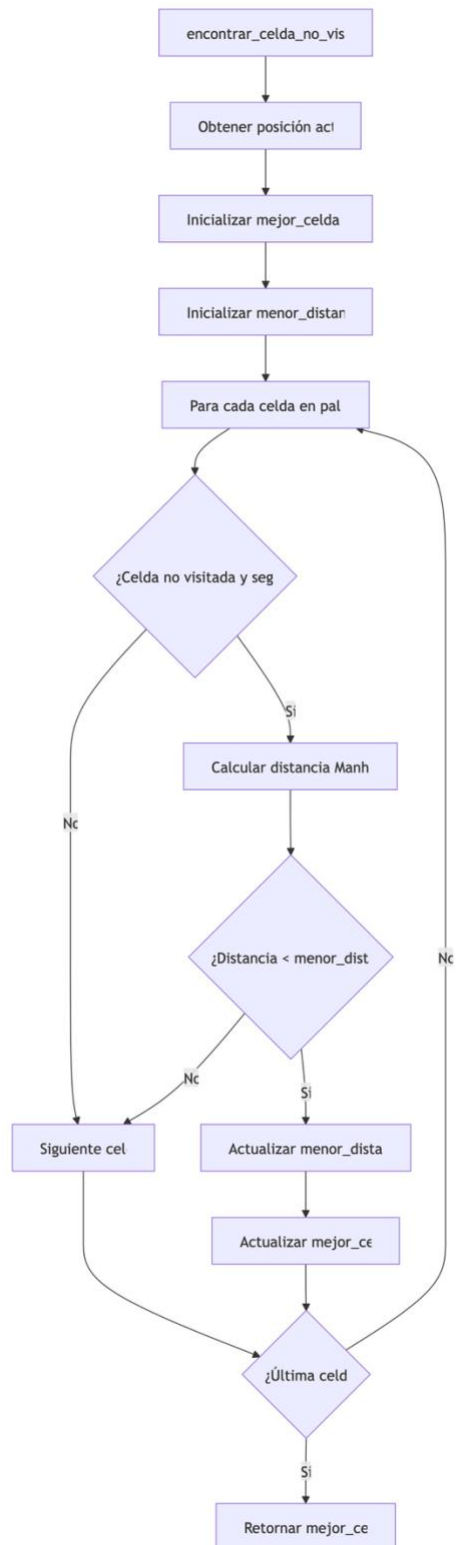


Figura A17. Diagrama de flujo de la función encontrar_celda_no_visitada_cercana. Elaboración propia.

Figura A18. Diagrama de flujo de la función decidir_siguiente_movimiento. Elaboración propia.

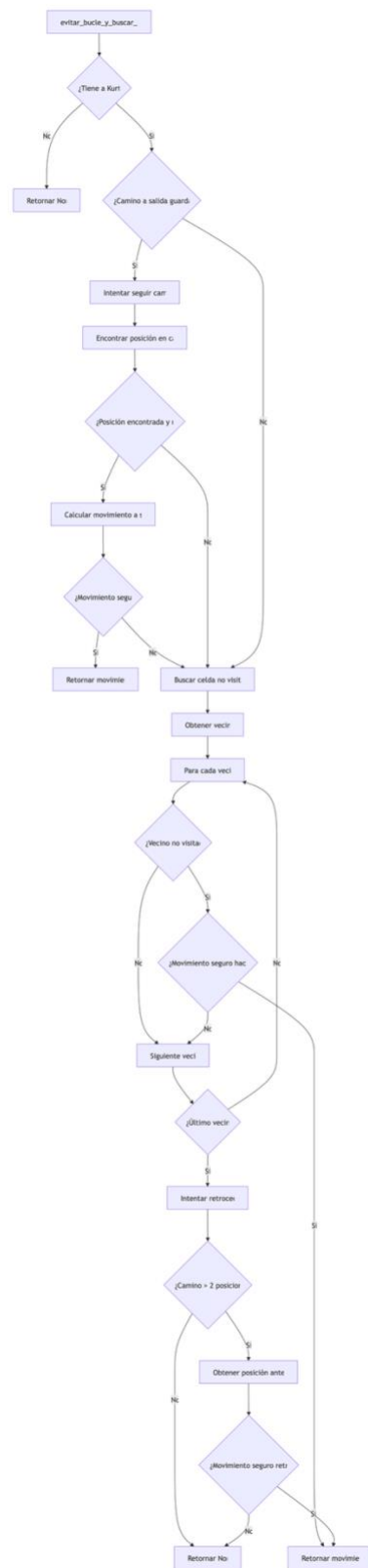


Figura A19. Diagrama de flujo de la función evitar_bucle_y_buscar_salida. Elaboración propia.

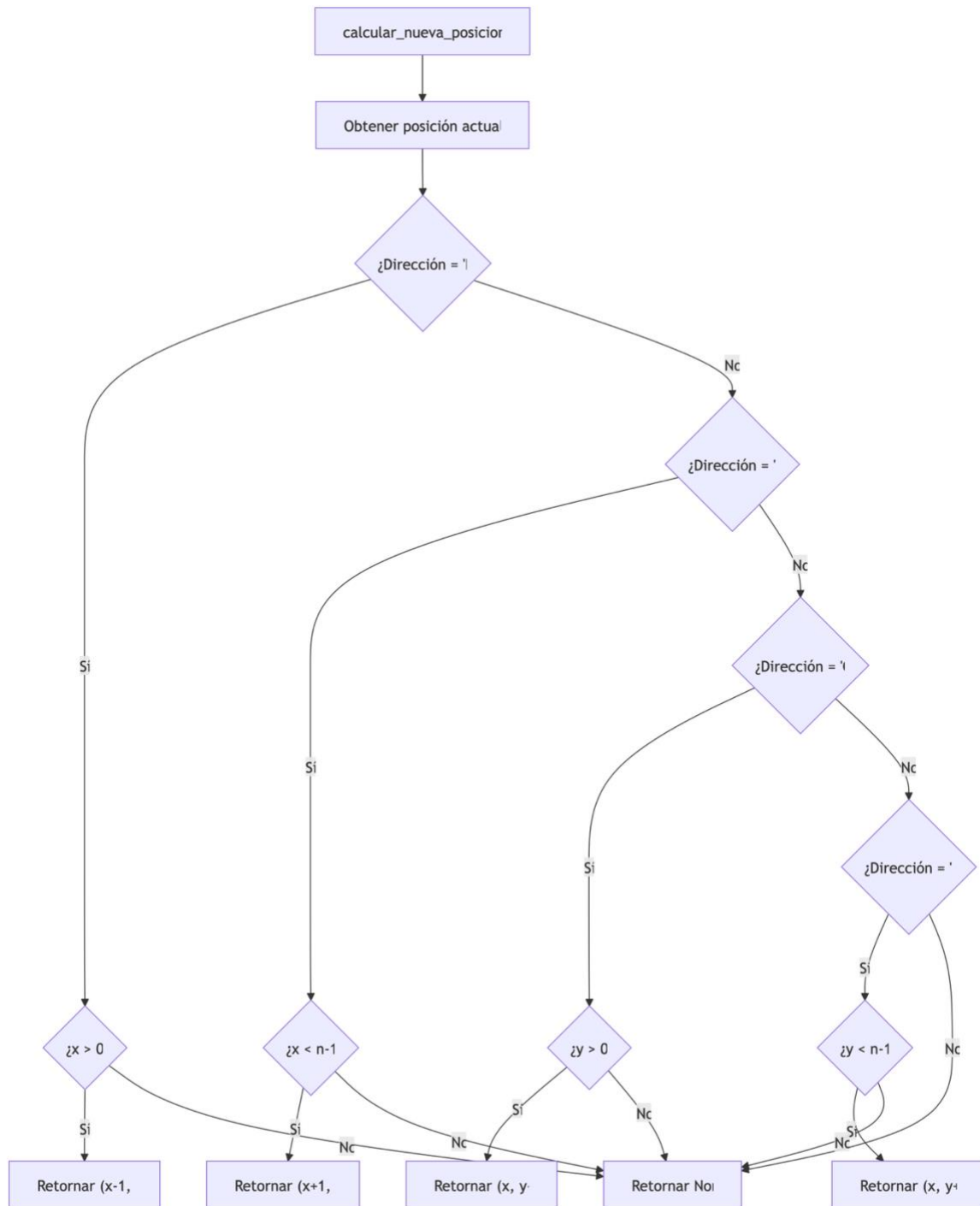


Figura A20. Diagrama de flujo de la función calcular_nueva_posicion. Elaboración propia.

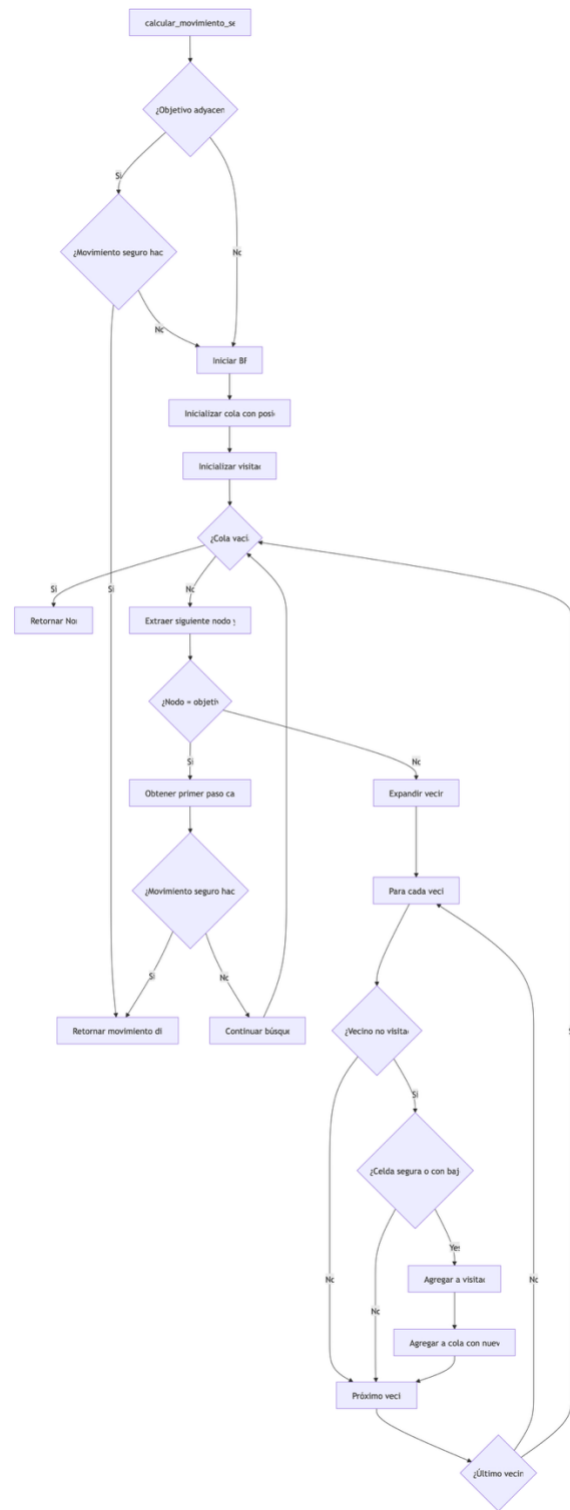


Figura A21. Diagrama de flujo de la función calcular_movimiento_seguro_hacia. Elaboración propia.

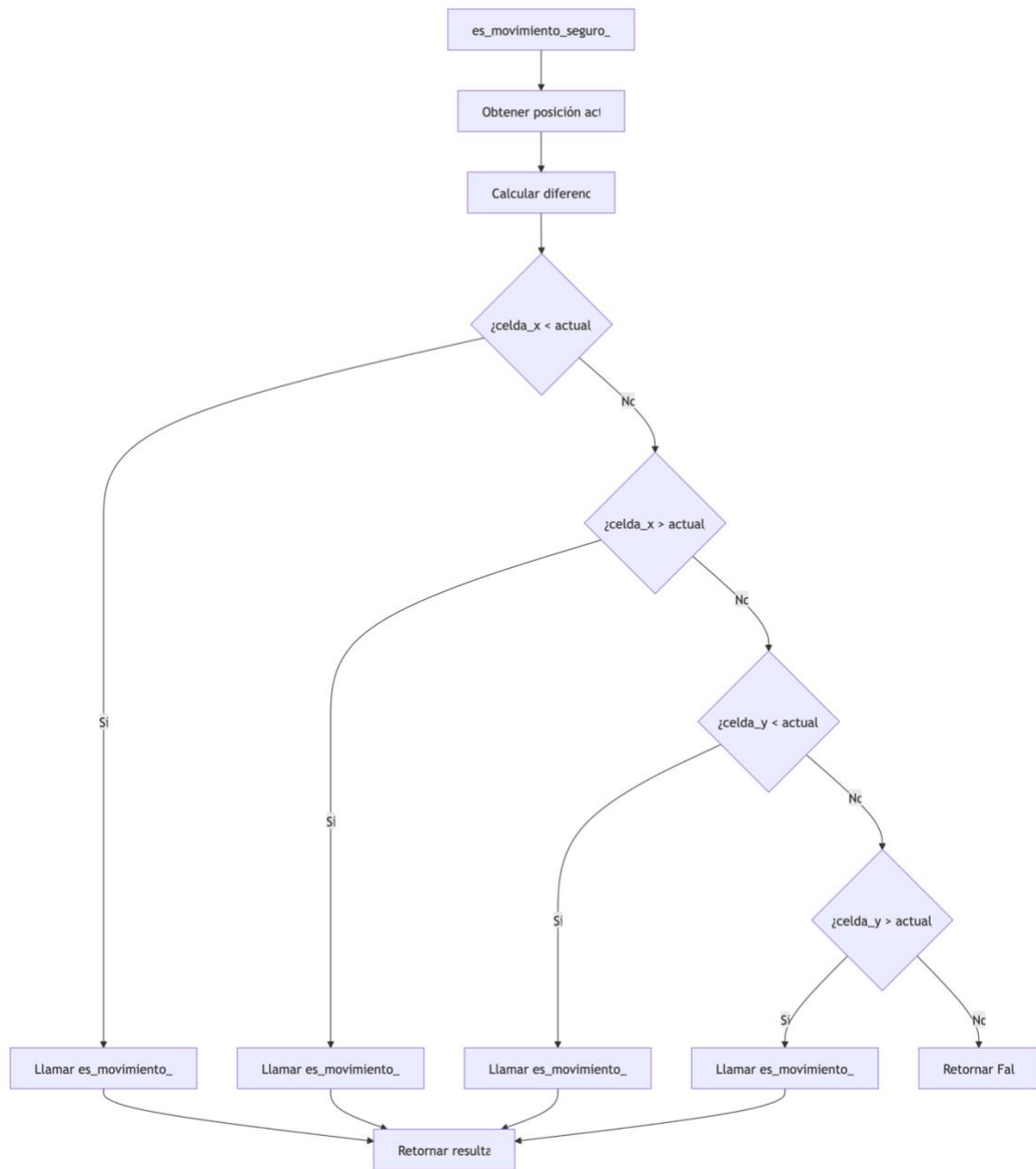


Figura A22. Diagrama de flujo de la función es_movimiento_seguro_hacia. Elaboración propia.

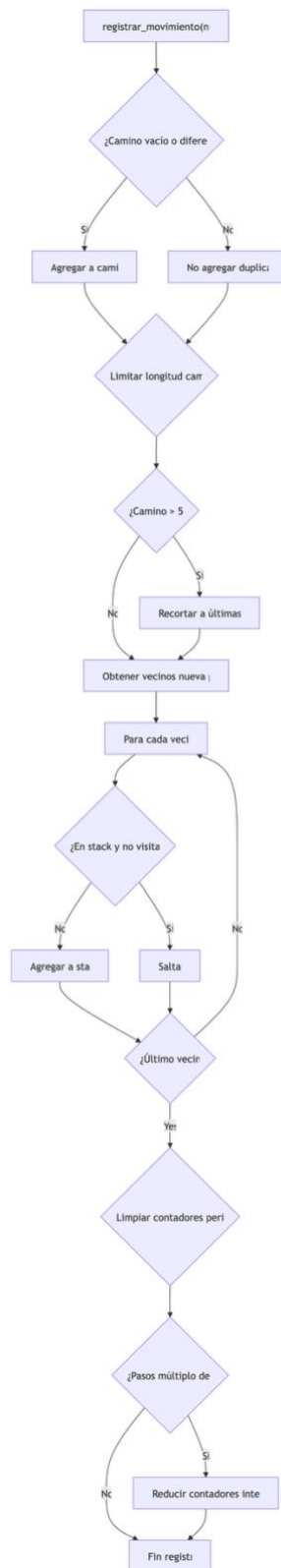


Figura A23. Diagrama de flujo de la función registrar_movimiento. Elaboración propia.

- class RiverMDP:

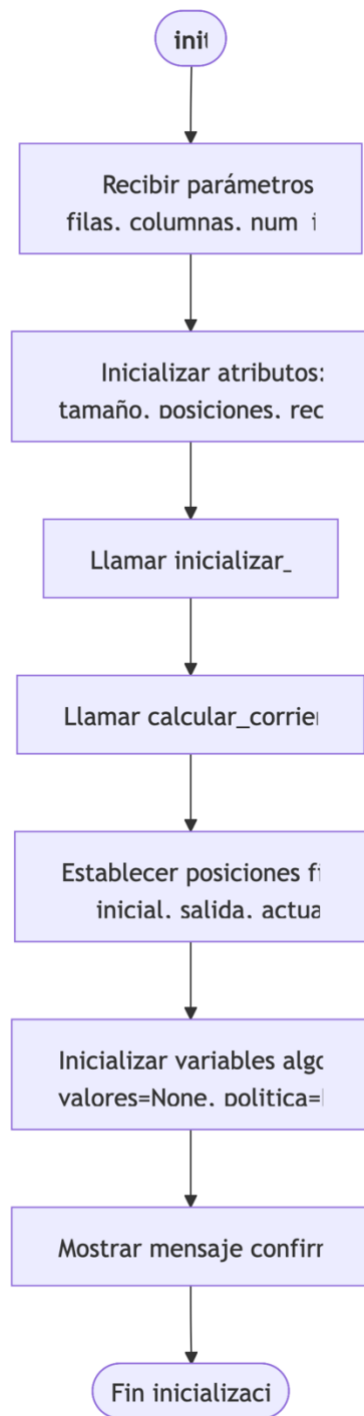


Figura A.24 Diagrama de flujo de la inicialización de la clase RiverMDP. Elaboración propia.

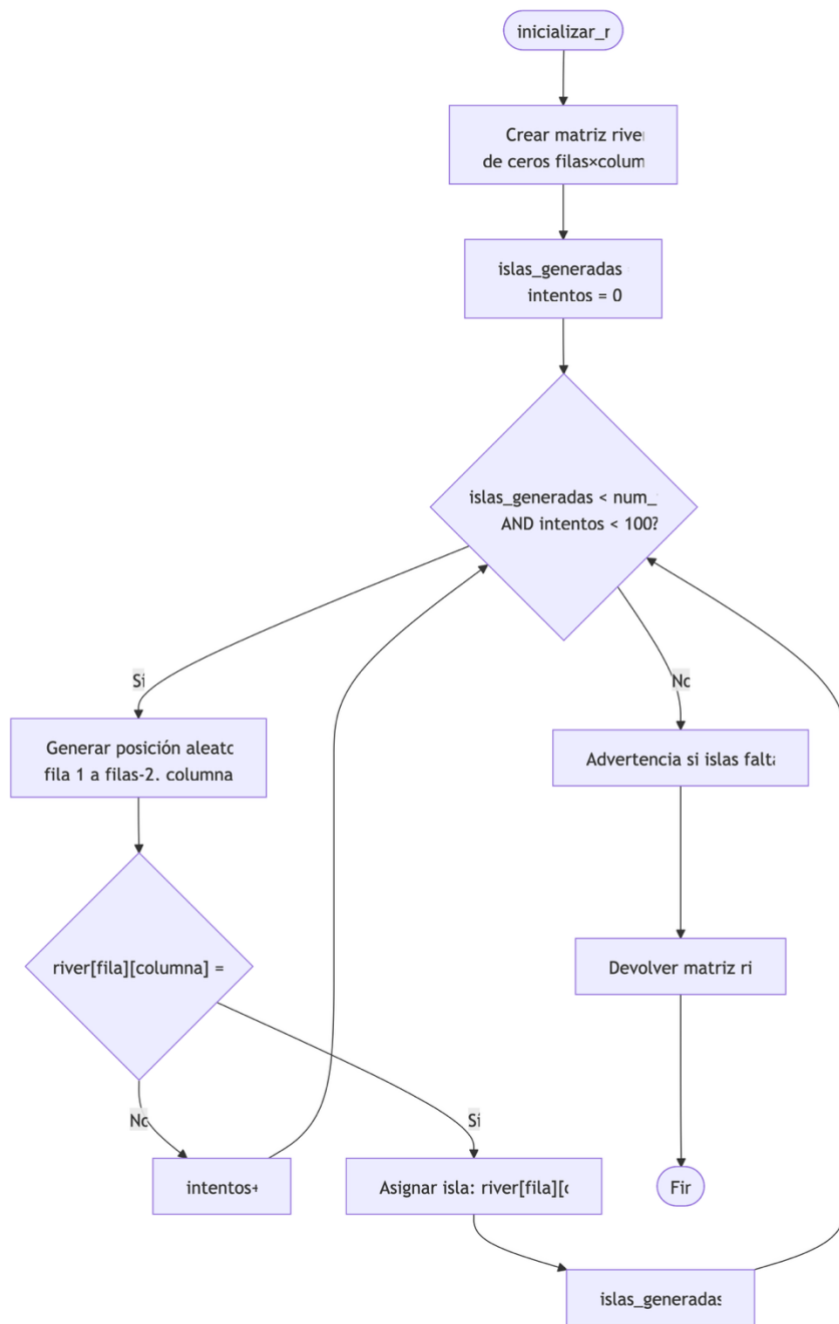


Figura A25. Diagrama de flujo de la función inicializar_rio. Elaboración propia.

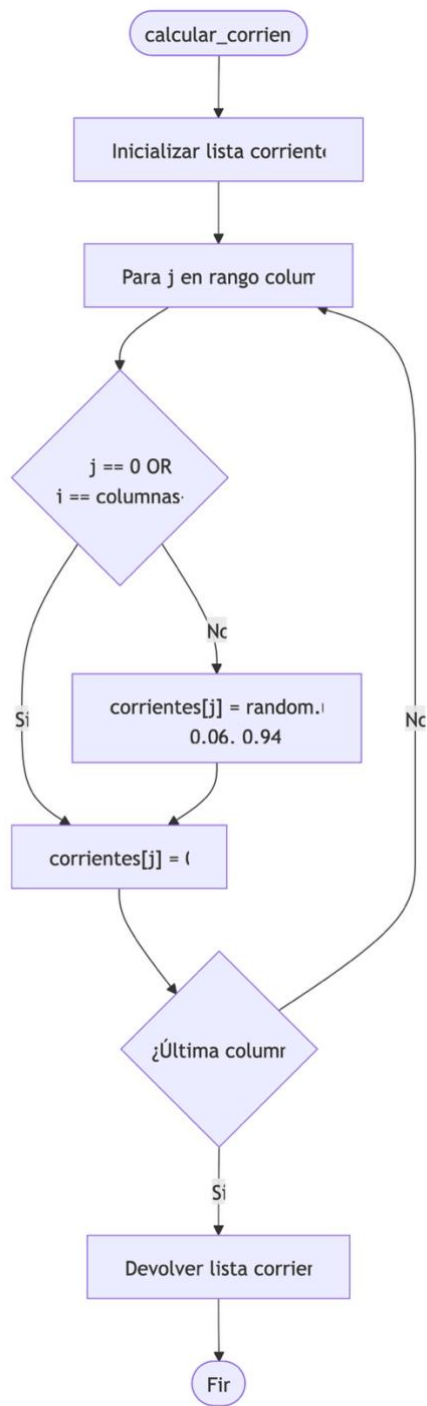


Figura A26. Diagrama de flujo de la función calcular_corrientes. Elaboración propia.

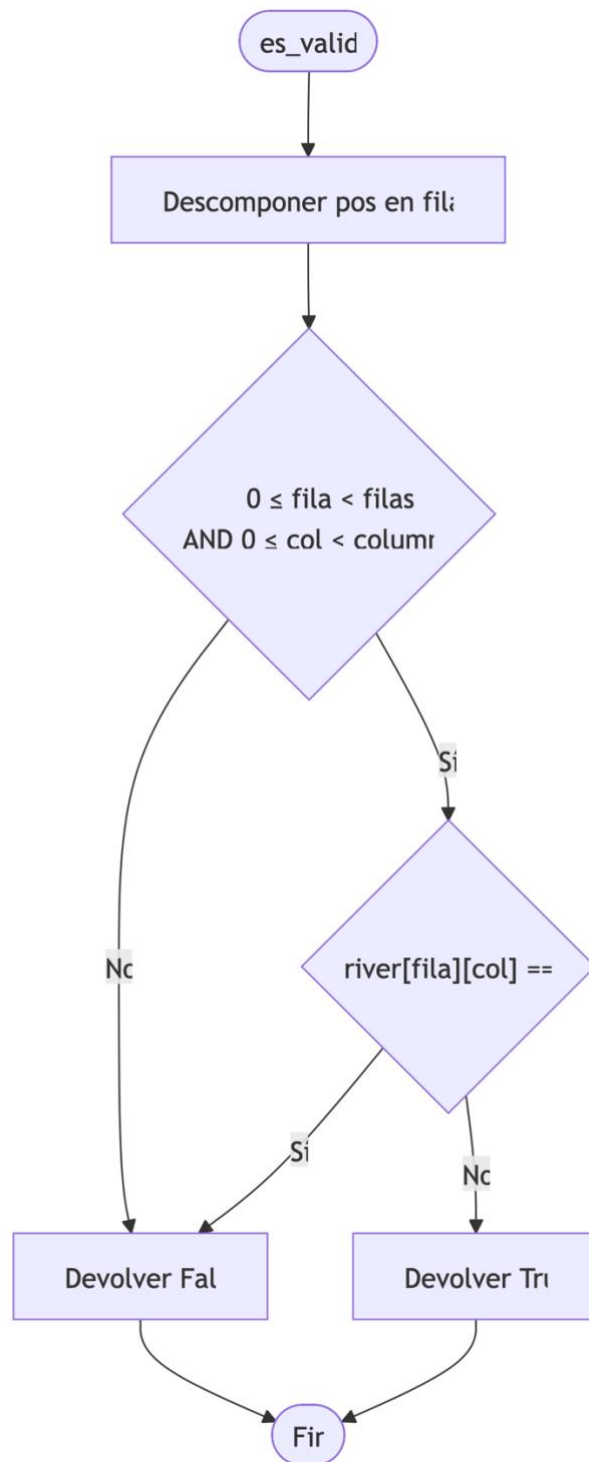


Figura A27. Diagrama de flujo de la función es_valida. Elaboración propia.

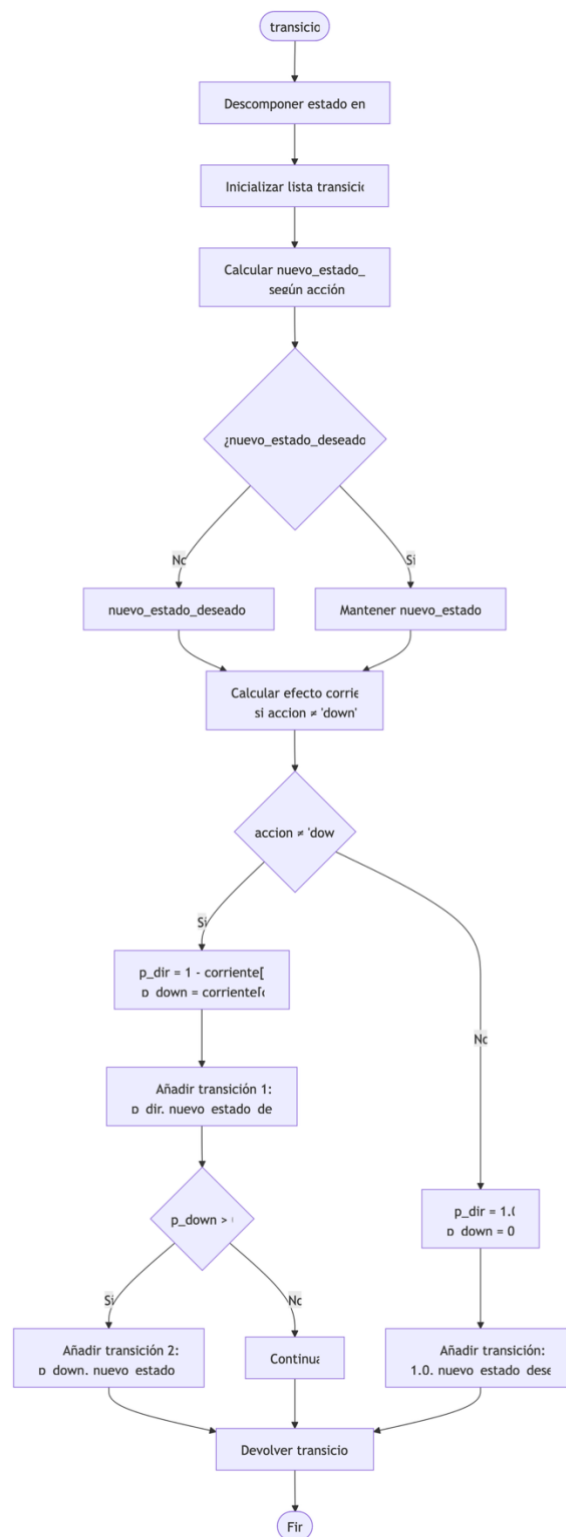


Figura A28. Diagrama de flujo de la función transicion. Elaboración propia.

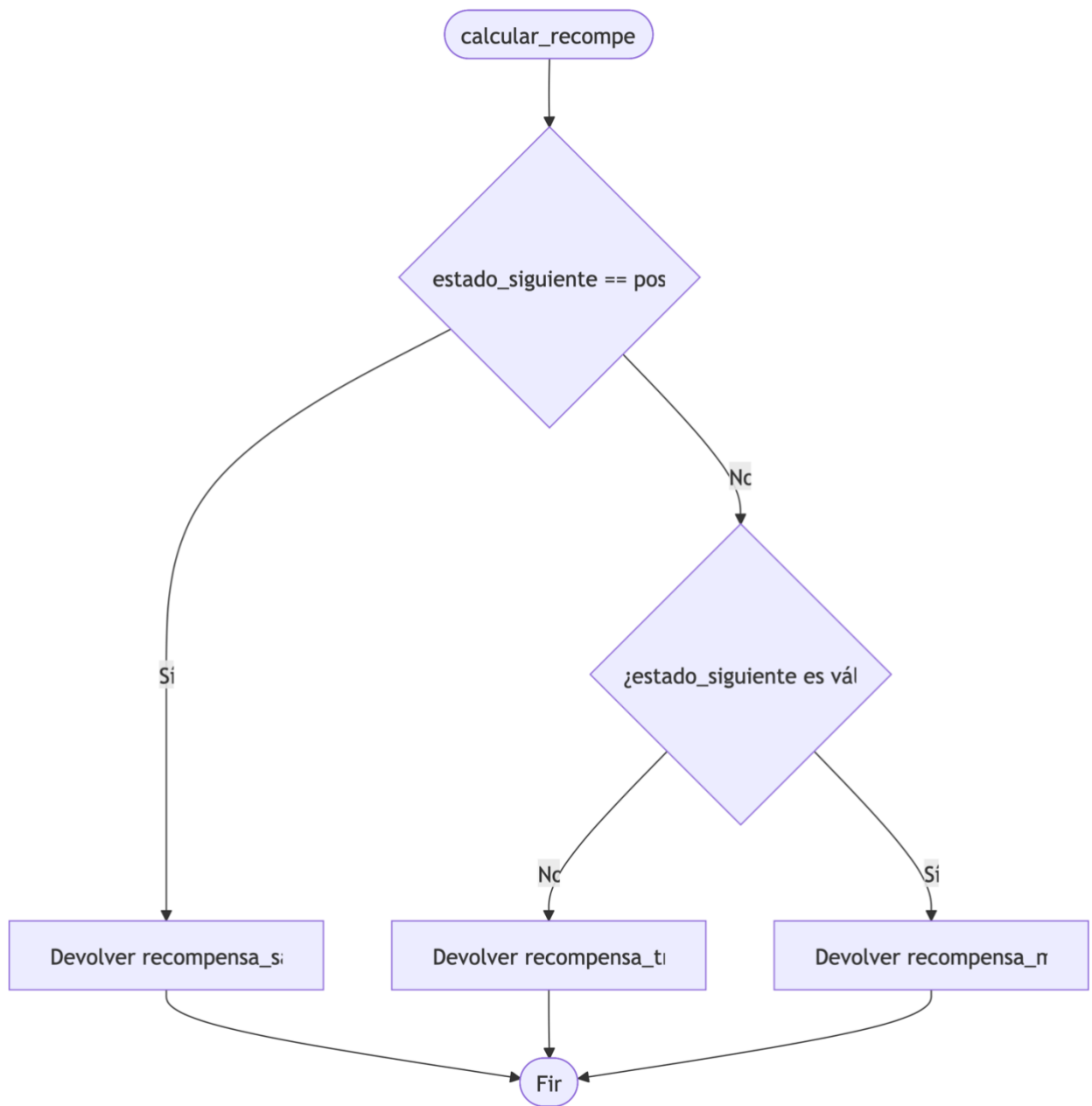


Figura A29. Diagrama de flujo de la función calcular_recompensa. Elaboración propia.

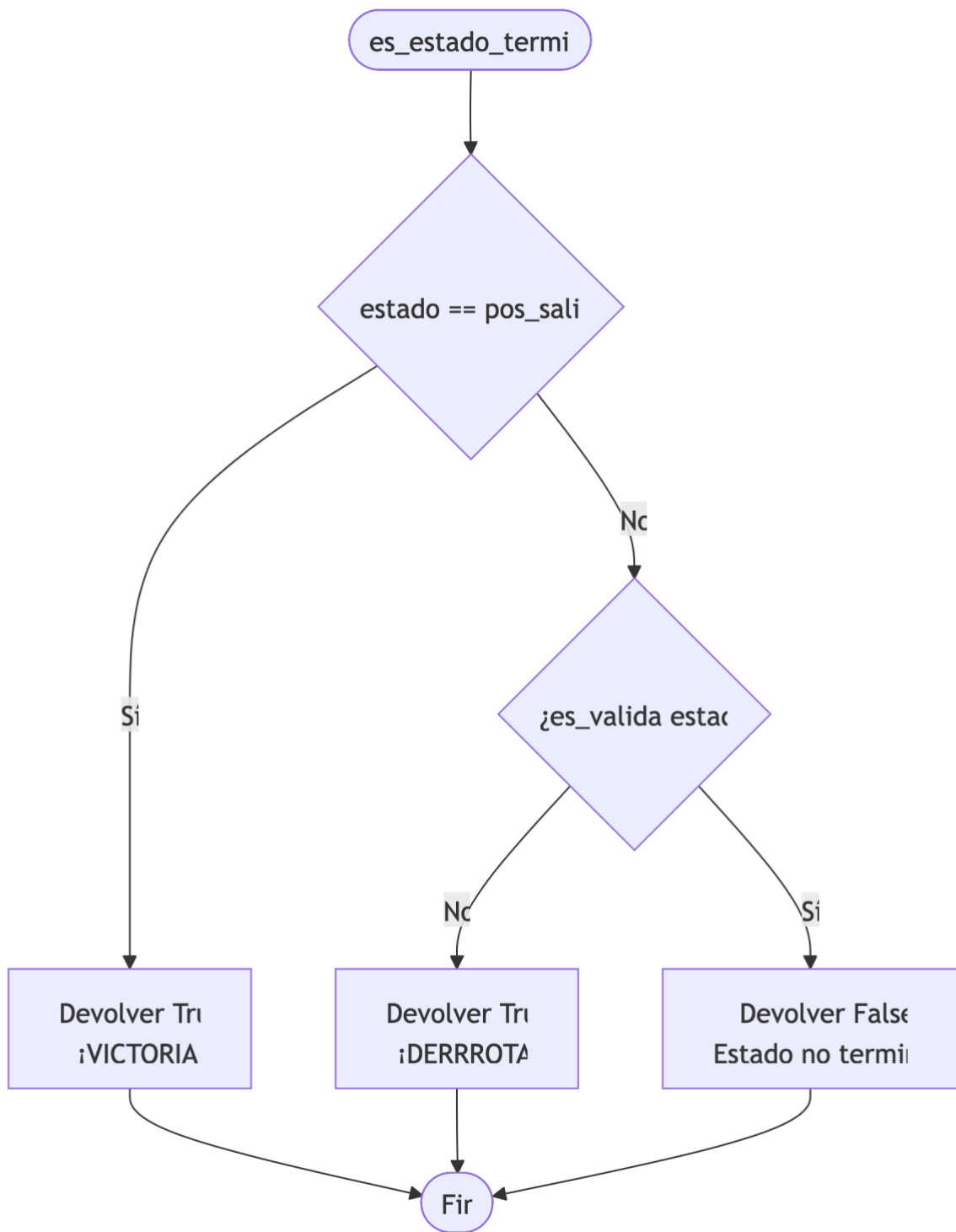


Figura A30. Diagrama de flujo de la función es_estado_terminal. Elaboración propia.

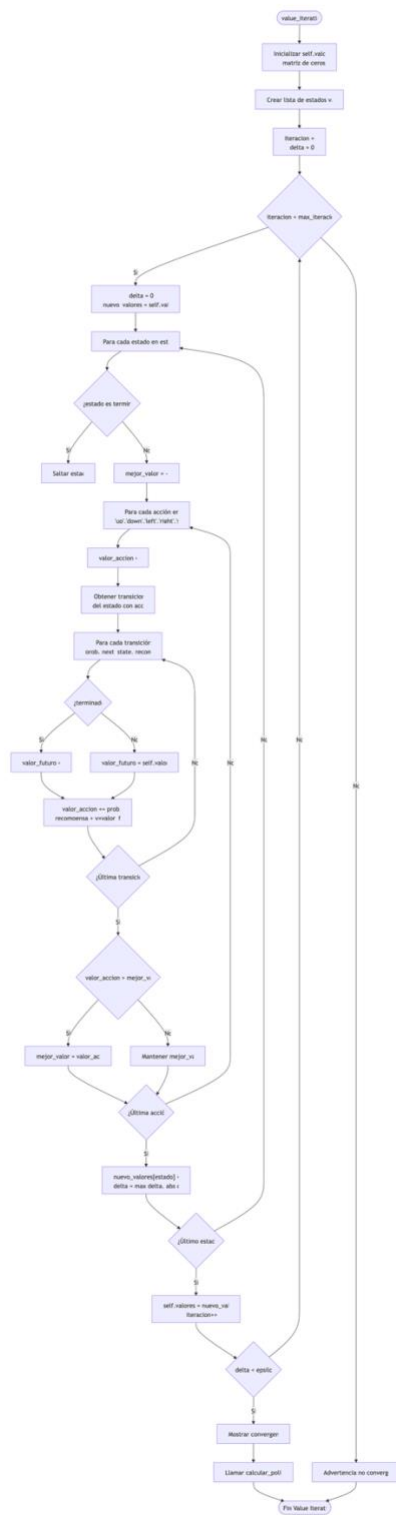


Figura A31. Diagrama de flujo de la función value_iteration. Elaboración propia.

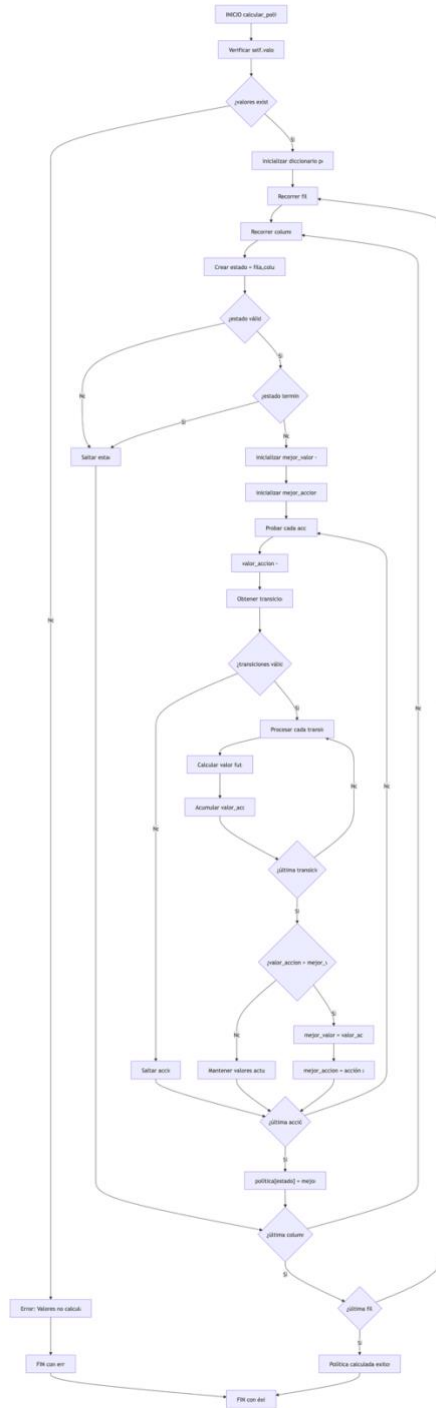


Figura A32. Diagrama de flujo de la función calcular_politica. Elaboración propia.

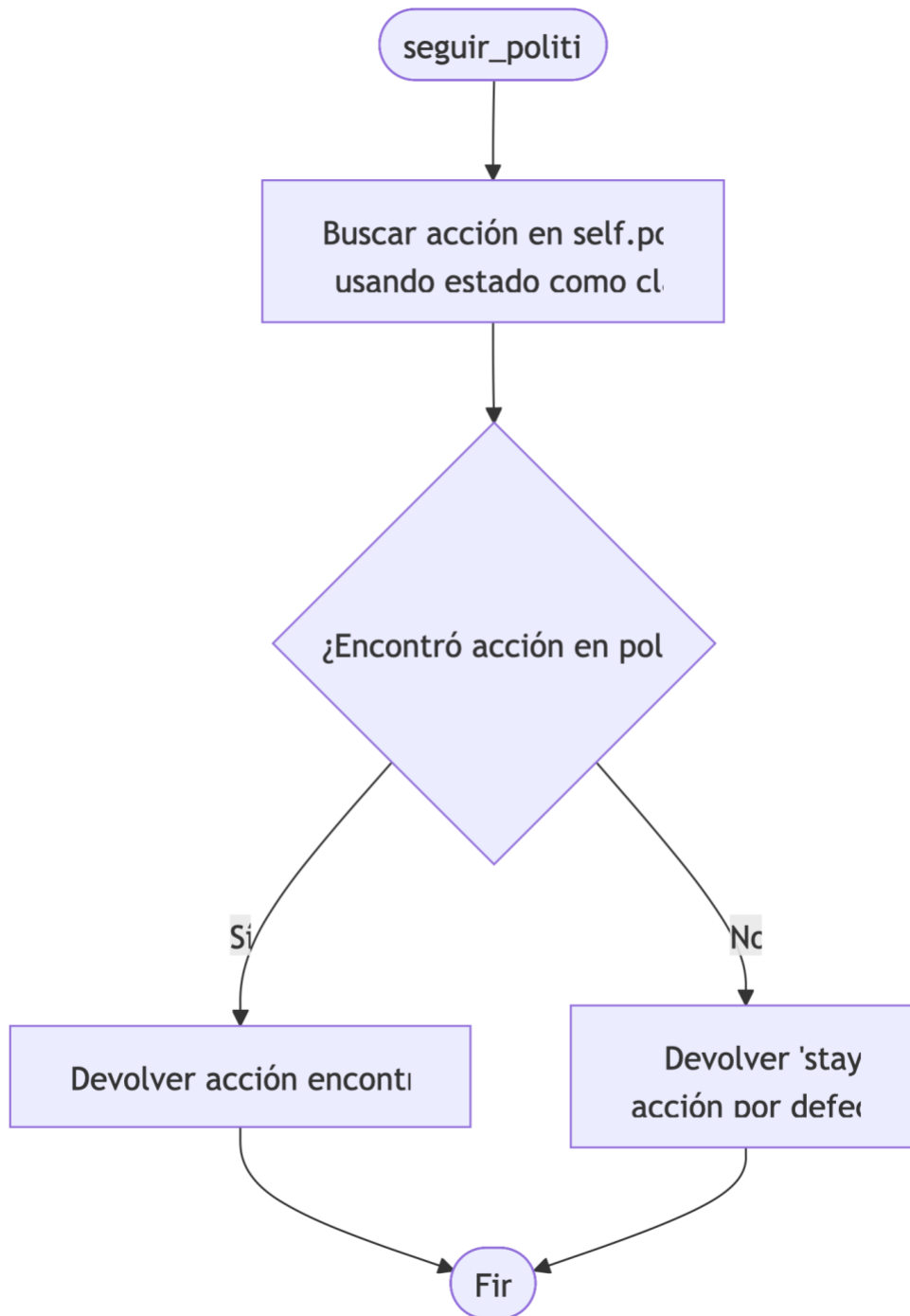


Figura A33. Diagrama de flujo de la función seguir_politica. Elaboración propia.

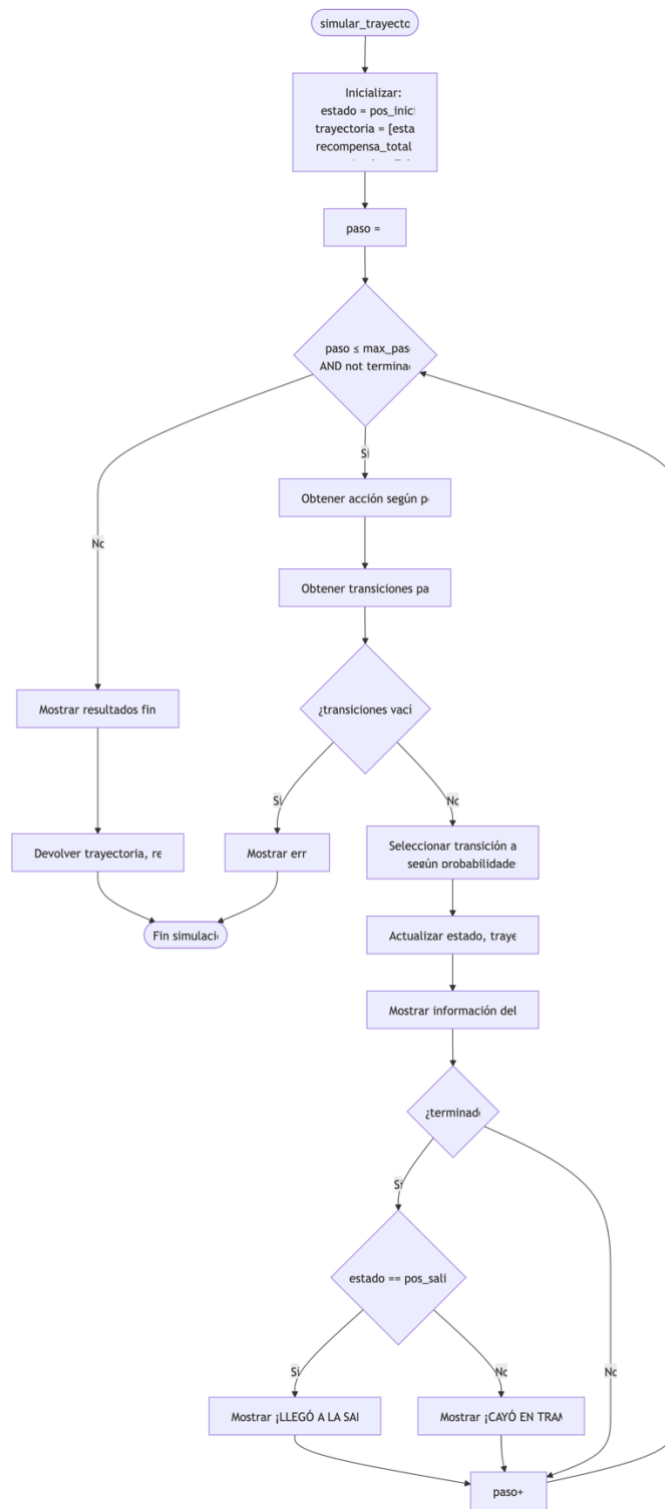


Figura A34. Diagrama de flujo de la función `simular_trayectoria`. Elaboración propia.

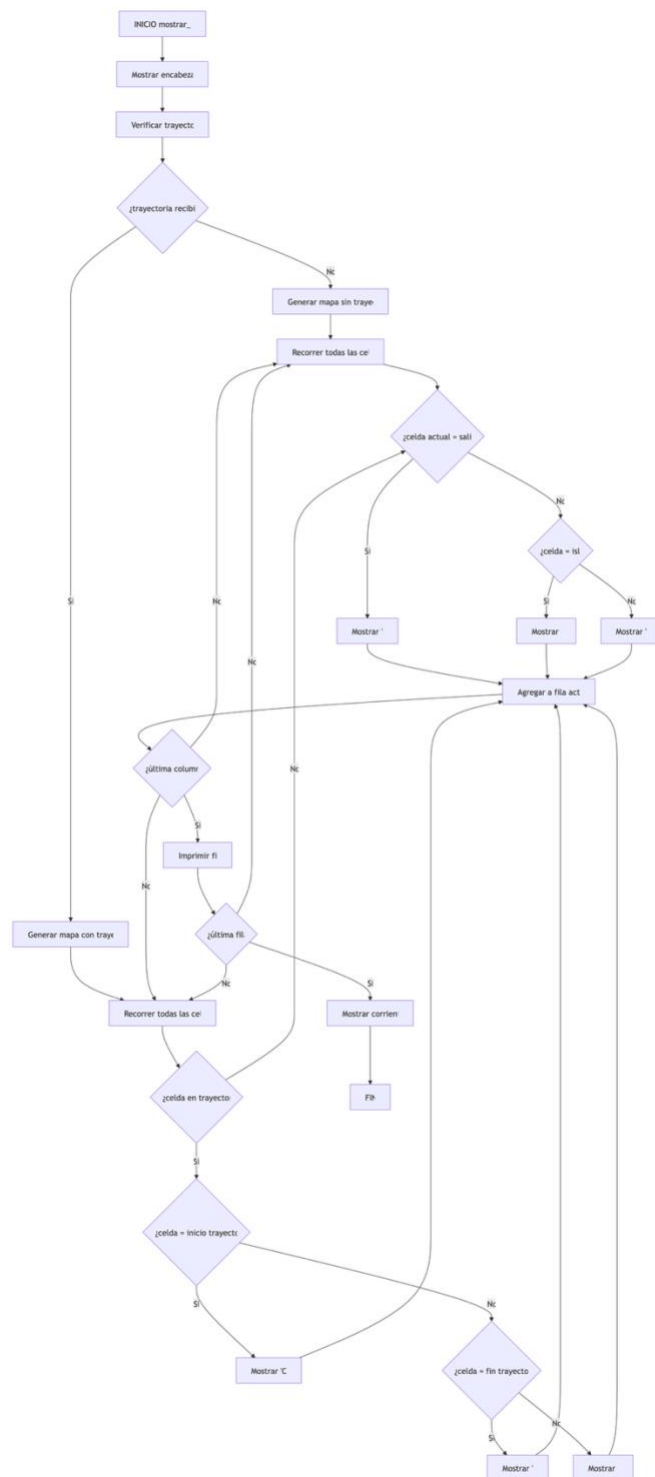


Figura A35. Diagrama de flujo de la función mostrar_rio. Elaboración propia.

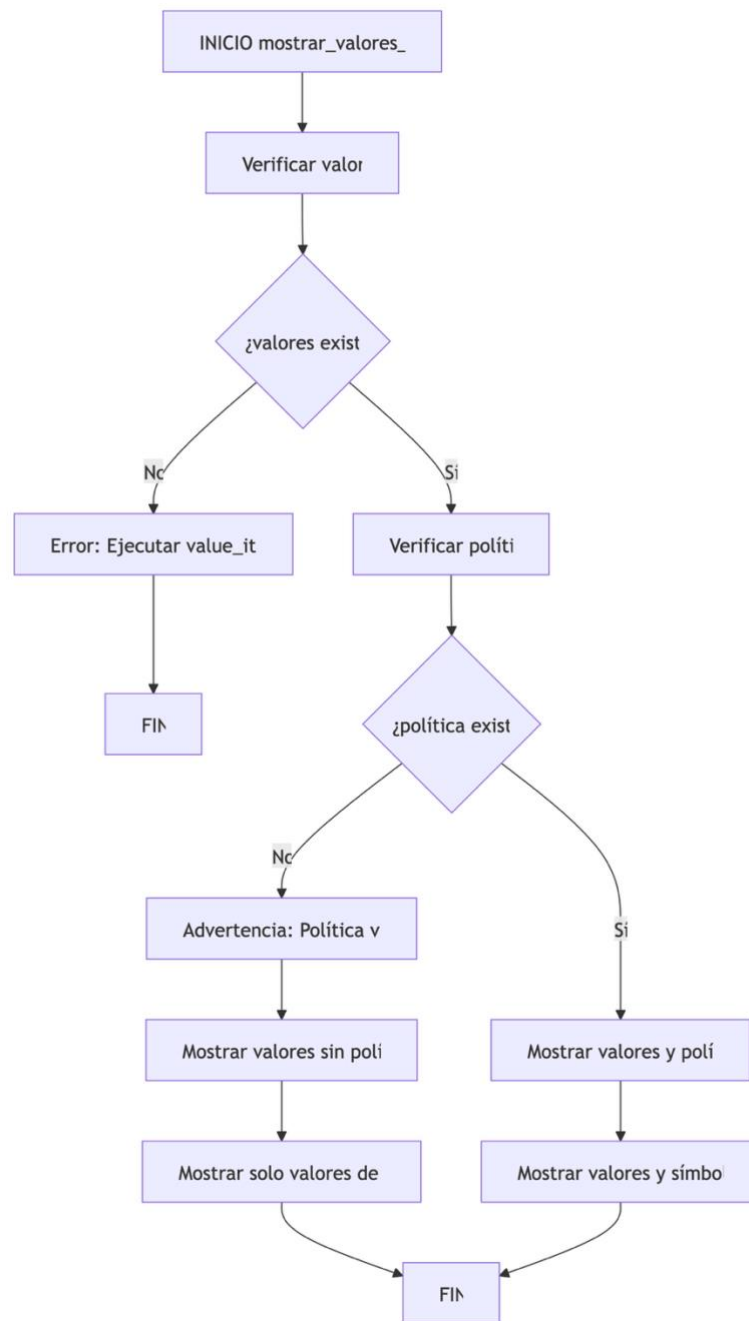


Figura A36. Diagrama de flujo de la función `mostrar_valores_politica`. Elaboración propia.